



成都大学  
CHENGDU UNIVERSITY

# 本科毕业设计（论文）

|        |                 |    |        |
|--------|-----------------|----|--------|
| 题    目 | 基于多模态大模型的       |    |        |
|        | 瑜伽动作评估系统        |    |        |
| 学    院 | 计算机学院           |    |        |
| 专    业 | 计算机科学与技术        |    |        |
| 学生姓名   | 曹黎骏             |    |        |
| 学    号 | 202210305212    | 班级 | 计科 1 班 |
| 指导教师   | 张健伟             | 职称 | 讲师     |
| 完成时间   | 2026 年 3 月 23 日 |    |        |

## 原创性声明

本人郑重声明：本人所呈交的毕业设计（论文），是在指导老师的指导下独立进行研究所取得的成果。毕业设计（论文）中凡引用他人已经发表或未发表的成果、数据、观点等，均已明确注明出处。除文中已经注明引用的内容外，不包含任何其他个人或集体已经发表或撰写过的科研成果。对本文的研究成果做出重要贡献的个人和集体，均已在文中以明确方式标明。

本声明的法律责任由本人承担。

论文作者签名：曹黎骏

日 期：2026 年 3 月 23 日

## 关于使用授权的声明

本人在指导老师指导下所完成的毕业设计（论文）及相关的资料（包括图纸、试验记录、原始数据、实物照片、图片、录音带、设计手稿等），知识产权归属成都大学。本人完全了解成都大学有关保存、使用毕业设计（论文）的规定，本人授权成都大学可以将本毕业设计（论文）的全部或部分内容编入有关数据库进行检索，可以采用任何复制手段保存和汇编本毕业设计（论文）。如果发表相关成果，一定征得指导教师同意，且第一署名单位为成都大学。本人离校后使用毕业设计（论文）或与该论文直接相关的学术论文或成果时，第一署名单位仍然为成都大学。

本毕业设计（论文）内容不涉及国家秘密、商业秘密及其他需要保密的内容，适用于上述授权。

论文作者签名：曹黎骏

日 期：2026 年 3 月 23 日

指导教师签名：

日 期：

# 基于多模态大模型的瑜伽动作评估系统

专业：计算机科学与技术

学号：202210305212

学生：曹黎骏

指导教师：张健伟

**摘要：**本文所研究的居家瑜伽练习动作评估问题是，很多居家瑜伽练习者跟着练瑜伽视频能跳动作，但是对动作关节、身体对齐、动作稳定是否做到位是没有办法正确反馈的。正位与控制是瑜伽训练中重要的一环，如果肩、髋、膝、脊柱长期处在不合理的正位中，训练的效果会因肢体的问题而减缩，运动损伤的风险也会愈加大，线下教练对动作的纠正是即时可以反馈的，但是会受到时间、位置、费用等条件的约束，而居家练习瑜伽的更多是居家练习者，更需要一套可以反复使用、反馈相对清晰，并且易于留存的辅助评估工具。

围绕这一需求，本文设计与实现了一套基于多模态大模型的瑜伽动作评估系统，其采用前后端分离的架构进行实现：前端通过 HTML、CSS 和原生 JavaScript 实现注册登录，视频上传，任务轮询与结果展示的功能；以 Flask 框架为后端框架为其提供 RESTful 接口，并辅以 SQLite 数据库来储存相应的用户信息、视频任务、评估记录与系统日志。算法上，以采用 MediaPipe Pose 姿态估计算法为底层姿态提取工具，提取视频的 33 个身体关键点后，通过 OpenCV 几何计算和时序统计等方式计算出关节角度、对身体对等的偏差、动作稳定程度等结构化特征。

在评估方法上，本文并未完全移交给外部模型打分的过程，系统保留了本地规则评估模块，由其根据动作标准库完成规则执行的基础打分和问题定位，再将关键帧、规则结果、结构化特征发送给网上的千问多模态模型，由其帮助生成更加自然的反馈文本和改善建议。这么做可以使得评估结果尽量可解释同时也可以在网络服务不太稳定的时候提供降级的结果。系统中先后经历了 Python 原型验证、Rust 后端试做、Flask 工程化落地三个阶段，目前已在联调中先后发现角色字段不一致、任务同步阻塞、SQLite 锁冲突、数据库触发器错误、空对象引用、代理变量干扰模型请求数据的问题。

最终实现系统已经完成了注册、登录、视频上传、后台异步处理、状态查询、结果回显、历史记录留存等主要链路，测试亦表明了系统可以在本地环境下完成完整的视频输入和评估输出流程，且至少在部分异常场景下保证基本可用。姿态估计算法本身并非本文主要工作，仅仅作为任务输入端的一种实现形式，重点还是在于把视觉分析、规则推断、多模态反馈、Web 工程实现整合成一套可以运行、可以验证、也可以继续扩展的毕设原型。

**关键词：**瑜伽动作评估；多模态大模型；Flask 框架；前后端分离；姿态估计

# Yoga Action Assessment System Based on Multimodal Large Language Models

Major: Computer Science and Technology

Student ID: 202210305212

Student: Cao Li Jun

Instructor: Zhang Jian Wei

**Abstract:** This thesis addresses a practical problem in home yoga training: learners may follow a pose video, but they still cannot easily judge whether their movement is aligned well enough. To deal with this issue, the project implements a yoga action assessment system that combines pose estimation, rule-based analysis, and multimodal language feedback.

The system uses a frontend-backend architecture. The frontend is built with HTML, CSS, and vanilla JavaScript for upload, polling, and result display. The backend is implemented with Flask framework, and SQLite database is used to store user data, task records, assessment results, and logs. In the analysis stage, MediaPipe Pose algorithm extracts 33 body landmarks from video frames, and OpenCV geometric calculations are then used to derive joint angles, alignment deviation, and motion stability features.

For evaluation, the project keeps a local rule-based scoring mechanism as the baseline and uses the Qwen multimodal model to convert structured errors and keyframe information into readable corrective suggestions. This design also makes fallback possible when the external model service is unstable. During development, several engineering issues were handled, including API field mismatch, SQLite locking, blocking processing, and proxy interference. The final system can complete the main assessment workflow on a local machine and generate usable feedback for yoga practice.

**Key words:** Yoga Action Assessment; Multimodal Large Language Model; MediaPipe; Flask; Frontend-Backend Separation; Pose Estimation

# 目录

|          |                                      |           |
|----------|--------------------------------------|-----------|
| <b>1</b> | <b>绪论 .....</b>                      | <b>1</b>  |
| 1.1      | 课题背景 .....                           | 1         |
| 1.2      | 国内外研究现状 .....                        | 1         |
| 1.3      | 面临的问题与挑战 .....                       | 3         |
| 1.4      | 主要工作内容 .....                         | 4         |
| 1.5      | 论文组织结构 .....                         | 4         |
| <b>2</b> | <b>系统需求 .....</b>                    | <b>5</b>  |
| 2.1      | 系统角色 .....                           | 5         |
| 2.2      | 系统功能 .....                           | 6         |
| 2.2.1    | 用户管理 .....                           | 6         |
| 2.2.2    | 视频上传 .....                           | 7         |
| 2.2.3    | 姿态多维特征提取 .....                       | 7         |
| 2.2.4    | 混合智能评估 .....                         | 7         |
| 2.2.5    | 结果展示 .....                           | 8         |
| 2.2.6    | 历史记录留存 .....                         | 8         |
| 2.3      | 系统性能 .....                           | 8         |
| <b>3</b> | <b>系统的设计 .....</b>                   | <b>10</b> |
| 3.1      | 系统体系结构 .....                         | 10        |
| 3.1.1    | 四层架构的职责界定 .....                      | 10        |
| 3.1.2    | 分层架构的工程化优势 .....                     | 13        |
| 3.2      | 系统运行环境配置 .....                       | 15        |
| 3.3      | 关键核心链路的技术实现 .....                    | 16        |
| 3.3.1    | 非阻塞式媒体上传与守护线程分配 .....                | 16        |
| 3.3.2    | 基于 MediaPipe BlazePose 的空间特征提取 ..... | 17        |
| 3.3.3    | 构建规则专家评估系统 .....                     | 19        |
| 3.3.4    | 基于多模态大模型的认知转换与信息提取 .....             | 21        |
| 3.3.5    | 数据库争用锁死与读写安全区隔离 .....                | 25        |
| 3.3.6    | 作用域逃逸引发的数据库触发器失效 .....               | 25        |
| 3.3.7    | 空值对象引用引发的雪崩 .....                    | 26        |
| 3.3.8    | 前端核心模块实现与优化 .....                    | 27        |
| 3.3.9    | 视频处理与内存管理优化 .....                    | 27        |

|       |                         |    |
|-------|-------------------------|----|
| 3.4   | 数据库设计细节 .....           | 27 |
| 3.4.1 | 持久化存储设计原则 .....         | 27 |
| 3.4.2 | 数据库实体与核心关系拓扑 .....      | 28 |
| 3.5   | 系统控制流与时序设计 .....        | 29 |
| 4     | 系统测试 .....              | 34 |
| 4.1   | 测试环境与总体测试策略 .....       | 34 |
| 4.2   | 核心功能集成测试的开展与剖析 .....    | 34 |
| 4.2.1 | 安全模块与角色约束链测试 .....      | 34 |
| 4.2.2 | 非阻塞上传与心跳轮询调度测试 .....    | 35 |
| 4.2.3 | 极限状态下的多模态认知链路试错测试 ..... | 35 |
| 4.3   | 研发踩坑与真实故障根因重现分析 .....   | 35 |
| 4.3.1 | 隐形代理劫持导致千问大模型静默阻断 ..... | 35 |
| 4.3.2 | 空值对象引用引发的雪崩 .....       | 36 |
| 4.4   | 多维度测试结果宏观总结 .....       | 36 |
|       | 结论 .....                | 39 |
|       | 参考文献 .....              | 39 |
|       | 致谢 .....                | 42 |

## 1 绪论

### 1.1 课题背景

近几年，居家健身因其便利性和经济性而越来越普遍。瑜伽也从一项兴趣爱好，变成很多人长期要坚持的一项日常训练。它和以前那种以心肺负荷为主的锻炼方式不同，瑜伽更讲究动作的完成度，也更讲究身心的控制程度。练习者不仅要把体式摆出来，还要注意肩、髋、膝、踝、脊柱是否在合理的点位上，呼吸和发力顺序是否在配合着。如果基础条件都没有把握好，动作可能看起来和标准型没有多大出入，但是训练收益往往是有限的，时间一长反复也会诱发代偿和损伤。

在真实场景里，很多人的困难不是“跟不上动作”，而是“无法判断自己做得准不准”。初学者对身体角度、关节排列缺乏稳定感知。居家练习，很多时候都是看着教学视频做形状。镜子视角小，手机回放又很难拉慢放指指出问题所在。一个动作是膝盖超伸了，还是骨盆前倾了，或者肩膀发力次序出问题了，练习者自己很难分辨。而问题始终得不到纠正的话，在重复练习中就会固化错误姿势。

线下教练能给予及时的反馈，这是传统教学方式最直接的优势。但这样的方式也有一个现实的边界，课程时间要合二为一，场地并不总是便利，长期上课也要负担一笔不小的费用。更关键的是，很多纠正意见停留在课堂当下，当练习者回到家里以后，依旧缺少一套可以随时复看反复比较留下记录的量化依据。也正因为如此，居家场景并不只是一个缺“能看动作”的工具，而是缺一套“能说明哪里有问题、为什么有问题、怎样调整”的辅助系统。

而计算机视觉、姿态估计、多模态模型的发展，让这个问题有了可以落地实施的技术路线。工程实现上，系统使用 Flask 框架<sup>[1]</sup>构建后端接口，并采用 SQLite 数据库<sup>[2]</sup>保存用户、视频任务、评估记录与系统日志；算法链路上，MediaPipe 姿态估计算法<sup>[3]</sup>能够比较稳定地抽取出视频中的 33 个关键点，加上 OpenCV<sup>[4]</sup>几何计算，系统就可以获得输出的关节和角度、对齐、动作稳定性等指标；在反馈生成阶段，系统再将结构化特征和关键帧信息交给千问多模态模型<sup>[5]</sup>处理，使其具备把“数字结果”翻译成更贴近日常教学语言的反馈文本的机会。基于这个背景，本文将以“基于多模态大模型的瑜伽动作评估系统”作为毕设题目，完成了一套可以运行、能够联调、能够保存结果和支撑论文论证的工程原型。

### 1.2 国内外研究现状

动作识别、姿态评估是计算机视觉领域的重要研究方向。在该领域，围绕这个方向，其实已经研究了很久。Andriluka 等人<sup>[6]</sup>建立的 MPII 人体姿态数据集至今仍是该领域重要的基准测试资源。Wei 等人<sup>[7]</sup>提出的卷积姿态机通过多阶段细化有效提升了姿态估计精度。Chen 等人<sup>[8]</sup>提出的级联金字塔网络则在处理遮挡和多人场景方面表现优



异。Saraf 等人<sup>[9]</sup>对基于深度学习的人体活动识别方法进行了系统综述，总结了从骨架序列到时空图卷积等多种技术路线。Singh 等人<sup>[10]</sup>则探索了利用 LSTM 网络从姿态序列中进行动作识别的方法。这些研究为瑜伽动作识别与评估提供了重要的技术基础和方法参考。以前更多关注的是“人在做什么动作”。而后来更多关注的是“这个动作做得是否规范”。这两个问题看着很接近，但难易程度其实是有区别的。前者偏分类。而后者不仅要识别动作，还要解释偏差、定位问题和给出反馈，所以对算法、规则、输出形式的要求都更高。

如果只看底层姿态感知，现阶段已经有开发较为成熟的开源工具可以利用，早先的手段更多地依赖人为主观设计特征的图像方法和几何模型。他们对于背景复杂或人体遮挡情况或是光照不稳定等方面的鲁棒性往往较差，OpenPose<sup>[11]</sup>之后的多人姿态的检测能力有了较为明显的提升，但它对算力的需求也更高，普通的开发机部署与调试的成本也相对更多。Newell 等人<sup>[12]</sup>提出的堆叠沙漏网络和 Xiao 等人<sup>[13]</sup>提出的 Simple Baselines 方法在人体姿态估计领域取得了显著进展，其中 Simple Baselines 以其简洁的网络结构在精度和速度之间取得了较好平衡。而 Zheng 等人<sup>[14]</sup>的综述研究则系统梳理了深度学习时代人体姿态估计技术的发展脉络。本文调研发现，瑜伽动作识别方向也有学者开展过相关工作，如 Zhang 等人<sup>[15]</sup>和 Field 等人<sup>[16]</sup>分别从视频分析和姿态估计角度对瑜伽动作进行了识别与校正研究。而 MediaPipe 姿态估计算法<sup>[3]</sup>似乎更适合本文这种以本地验证与落地工程为主的课题，它基于 BlazePose 架构，输出 33 个关键点，模型的推理速度也适合在普通 PC 环境下部署，被广泛地应用在需要轻量化的人体姿态分析的场景。

再看评估层面，根据公开资料、开发记录和现有开源实践，相关方法大致可以分为下面三类：

#### （1）规则驱动评估

系统提取关键点后，再计算肘、膝、髋、肩等关节的角度，把结果与系统设定好的标准进行比较。它的优点很明显是实现了直接驱动。打分依据很直接，也便于解释扣分的原因。而它的缺点也能很明显地看出来。不同的练习者的体型、柔韧性、拍摄的角度等并不是统一的，一成不变的阈值很容易造成误判，反馈的文本通常比较生硬。

#### （2）端到端学习评估

这类方法一般收集带有评分标签的视频或骨架序列，再学习时空模型直接输出动作分值，它在复杂场景下有较好的整体拟合能力，但往往可解释性较差。用户看到分数后依然容易不知道是哪一个关节、哪一段动作或者哪一种发力方式出现问题。

#### （3）多模态联合评估

这个角度是近两年更多人关注的一个角度，尝试将视频关键帧，姿态序列和结构化特征同时交给多模态模型，然后输出自然语言的评价语。Papadopoulos 等人<sup>[17]</sup>在跨领域多模态概念检测方面的研究为多模态特征融合提供了重要参考，Horn<sup>[18]</sup>关于层次化姿态估计的综述研究则从另一个视角梳理了姿态感知技术的发展脉络。Rich 等人<sup>[19]</sup>在

3D 人体姿态估计与跟踪方面的工作进一步拓展了姿态分析的维度。其特点是表达能力更强，更贴合日常的教学场景，但会涌现更多新的问题，例如：提示语词设计，输出格式控制，接口稳定性问题，成本问题。

对本课题来说，更实际的目标不是做一个“能够识别出动作名称”的展示程序，而是把识别结果继续落到评分、解释和建议上。瑜伽场景对细节有着相当的要求，做了一个系统，对于用户来说，只能判断出他做的究竟是什么体式，对其中偏差出现在什么部位、应当怎样调整什么动作都无法解释的话，对真实的练习并没有什么帮助。在判断下，本文最终选择把姿态提取、几何规则分析和多模态反馈放到同一套系统里做，用工程的方式完成一次相对完整的尝试。

### 1.3 面临的问题与挑战

把一个“视频分析加模型反馈”的想法做成可交互、可演示、可保存结果的 Web 系统的难点其实不只在算法，很多问题在原型阶段看不到，等前端、后端、数据库、外部模型服务等全部搭起来后，就会出现新的工程压力。结合领域经验，主要有以下几方面的挑战：

#### （1）动作标准难以固定

同一个体式，不同练习者的身高、体量和柔韧性都会有差异。规则若是僵硬的，系统会轻易地把合理动作判断为错误动作。规则若是较弱的，评估又会失去压力。

#### （2）居家视频质量不稳定

居家拍摄视频往往存在光线阴暗、背景杂物多、机位随意和局部遮挡的情况，同时瑜伽肢体动作本身存在很多肢体交叠的情况，以上条件下关键点检测模型更容易漂移，且置信度更差。

#### （3）规则结果不等同于用户能看懂的反馈

几何计算能给出角度的偏差，但用户需要的不只是“差了多少度”，他想知道问题出在哪个部位、为什么会这样、练习时应该怎么调整，单靠规则判断很难把这个表达得足够自然。

#### （4）外部模型服务不可控

大模型接口可能存在代理变量、网络波动、请求超时、返回格式异常的扰动因素，实际项目中就遇到过联调时真实环境代理环境变量导致请求异常阻断的问题，如果系统没有容错和降级机制，就会把主流程拖住，变成单点故障。

#### （5）多模块联动容易暴露工程故障

视频解码、姿态识别、数据库写入、线程调度、前端轮询存在较为明显的耦合。开发记录中存在过上传同步阻塞、接口字段不一致、`no such column: user_id` 触发器错误，以及 `NoneType is not iterable` 等问题，可见整条链路真正稳定下来并不是一件容易的事。

把这些问题放在一起看，你会发现该课题真正难的不是某一个单点模块，而是串接

起来之后是否还能继续工作。也正是因为这样，该文在实现阶段关注的不是某一个局部指标，而是整条业务能否跑通，结果能否解释，异常能否兜底。

## 1.4 主要工作内容

本文在现有的姿态估计算法的基础上进行创新，将其融合成一个综合评估系统，重点在于系统集成、流程打通和工程实现。本文探讨了如何才能把已有的姿态提取方法、规则分析思路和多模态反馈能力组织成一套能运转的工程实践。围绕这个目的，本文完成的主要工作有：

### （1）用户需求梳理

根据居家练习场景进行用户需求梳理，梳理普通用户和管理员这两类角色权限边界、输入方式和结果呈现目标

### （2）关键点提取

采用 MediaPipe 姿态估计算法<sup>[3]</sup>完成对视频帧的人体关键点的提取，形成后续角提取和稳定性判定算法的数据支撑

### （3）特征计算模块

设计基于几何关系和时序统计计算的特征计算模块，对关键关节角度、左右对齐关系、动作波动情况进行计算量化

### （4）规则评估模块

构建本地规则评估模块，依托动作标准库实现基础打分、问题定位、降级输出

### （5）千问模型接入

接入千问模型，针对提示词、结构化返回、错误解析机制进行多轮调试，让反馈文本更贴近日常教学语言

### （6）业务链路实现

完成注册登录、上传视频、后台异步任务、轮询查询、结果回显、历史记录等主要业务链路

### （7）故障修复

定位并修复对联调中出现的真实故障，如前后端字段不匹配、角色字段映射冲突、数据库锁冲突、触发器列引用错误、空对象迭代异常、代理变量造成的模型请求失败

## 1.5 论文组织结构

论文整体按照“问题提出、需求拆解、系统设计、实验验证”来展开，其中：第一章为绪论，阐述课题的背景、相关研究现状、项目存在的主要问题和论文针对该问题的工作内容，第二章为系统需求阐述，从角色划分、功能模块、性能需求三个角度阐述系统需要解决什么问题，第三章主要对系统架构演进、运行环境、重要实现链路和实现的数据库设计、业务时序做了阐述，是本论文的实现重点，第四章对于提出并通过具体真实的联调对系统做了功能、异常与对故障的测试分析，正文之后依次为本文的结论部分、参考文献与致谢。

## 2 系统需求

### 2.1 系统角色

权限边界是否清晰，会直接影响到 Web 系统是否可以稳定地运行，所以需求分析第一步要把角色定义清晰。结合本项目的使用方式，系统内的角色主要有两类，一类是普通用户，一类是管理员，两者共享一套后端服务，但是能访问的页面、能进行的操作、能看见的数据范围都不一样。

普通用户：普通用户是系统的主要使用者，是整套业务流程的主要完成者。典型操作包括注册登录、瑜伽体式选择、训练视频上传、分析任务提交、处理进度查看、评估结束后的评分建议查看。还需要经常查看历史用来在前后不同时期做比较。对这类用户来说，功能多少不是需求重点，能否一目了然，等待过程是否可感知，失败原因能否说明白。换句话说，就是后台的处理过程前端必须做隐藏处理，呈现出来的东西必须是有帮助的练习信息。

管理员：管理员的职责更偏向于平台维护和运行监控。相较于普通用户，管理员既要能查看数据的统计，了解注册人数、评估次数、动作热度、平均分等平台层面的信息；同时也要能在必要时管理用户状态，如调整角色、禁用异常账号等。管理员不是多一个界面而已，而是意味着系统要对高权限操作进行更严格的认证和拦截。

这样的角色划分有两个好处，一是可以让普通用户界面保持尽量简洁，二是让后台管理与日常使用分离，减少权限混乱带来的安全风险。

从真实使用过程看，普通用户和管理员虽然共用同一套服务端程序，但二者关注的信息粒度完全不同。普通用户更关心单次练习是否顺利完成、分数是否容易理解、建议是否能指导下一次动作调整；管理员更关心系统是否稳定、任务是否堆积、异常是否能够追踪、用户数据是否存在越权访问风险。因此系统不能只在页面入口上区分角色，还需要在接口层和数据库查询层同时完成限制。比如普通用户访问历史记录时，只能读取自己名下的评估记录；管理员访问统计页面时，则需要读取汇总数据，但仍不应直接暴露用户密码哈希、模型原始响应等敏感内容。

角色设计还影响到错误处理方式。普通用户遇到失败时，需要看到的是“视频格式不支持”“分析超时，请稍后重试”这类可理解提示；管理员则需要在日志中看到更具体的异常类型、任务编号和发生阶段。若系统只返回底层异常文本，普通用户会难以理解；若系统只返回模糊提示，管理员又无法排查问题。因此本文在需求分析阶段就把角色和信息可见性联系起来考虑，保证同一个故障在不同角色侧有不同粒度的呈现。

此外，角色权限也是后续系统扩展的基础。若未来加入教师端或教练端，系统可以在现有普通用户和管理员之间增加新的中间角色，例如允许教练查看所负责学员的训练记录，但不允许修改全局配置和用户权限。当前项目虽然只实现了两类角色，但通过明

确的鉴权中间件和接口边界，已经为后续角色扩展预留了方向。

## 2.2 系统功能

系统功能并不是把几个页面机械地拼在一起，而是围绕“用户怎样完成一次完整评估”来组织业务流程。结合本文的实现目标，系统功能主要拆为用户管理、视频上传、姿态多维特征提取、混合智能评估、结果展示和历史记录留存六个部分，前后联系强，基本覆盖从输入到输出的完整链路。

从用户操作路径来看，一次完整评估通常经历五个阶段：用户先完成登录，随后选择或录制瑜伽动作视频，上传后等待系统分析，分析完成后查看评分与建议，最后在历史记录中保留本次结果用于后续对比。这个流程看似简单，但其中每一步都对应后台不同模块的协作。登录阶段依赖身份认证和权限判断；上传阶段依赖文件校验和任务创建；分析阶段依赖视频处理、姿态估计和规则计算；反馈阶段依赖模型调用和结果解析；历史阶段则依赖数据库持久化和查询接口。正因为流程跨越多个模块，系统功能设计不能只罗列页面按钮，而要围绕完整闭环组织。

功能模块之间也存在明显的依赖关系。没有用户管理，就无法把评估记录归属到具体用户；没有异步任务，就无法让视频分析在较长时间内保持前端可响应；没有规则评估，就无法在模型不可用时提供兜底结果；没有历史记录，系统就只是一种一次性演示工具，而不是可持续使用的训练辅助工具。因此本文在功能分析中把这些模块作为一条连续链路，而不是孤立功能点。这样的划分更符合项目实际，也能解释为什么某些看似“后台”的模块对用户体验同样重要。

在功能实现优先级上，本文优先保证主链路可跑通，再逐步补充管理端、日志、异常兜底和可视化细节。原因在于，毕业设计阶段最重要的是证明核心方案成立：用户上传视频后，系统能够完成分析并返回有意义的反馈。如果一开始把大量精力投入装饰性页面或复杂统计面板，主链路不稳定时反而难以支撑论文论证。因此系统开发采用了“先闭环、后完善”的策略，先完成最小可用流程，再围绕联调中暴露的问题持续补强。

### 2.2.1 用户管理

账户体系是系统运行的基础，所以用户管理模块首先要解决完整性和安全性问题。用户注册时，提交用户名或邮箱以及密码。后端要完成唯一性校验，还不能让密码以明文进入数据库，所以系统采用 Bcrypt 对密码做加盐哈希再保存，这样即便数据库数据泄露，原始密码也不容易被复原。

在鉴权机制上，由于系统采用前后端分离架构，鉴权机制不再使用传统的 Session/Cookie，而是采用 JSON Web Token (JWT)<sup>[20]</sup>作为统一身份凭证。用户登录成功后，后端生成 token，token 包含身份、角色、过期时间等信息。前端在后续请求中通过 Authorization 头携带 token，后端再进行校验与权限判断，这种方式更加符合 RESTful 接口风格，也便于前端独立部署。

开发过程中用户管理模块还暴露出典型的接口契约问题，即前后端对于角色字段命

名并不统一，例如前端为了显示更直观，对于学员角色使用 learner 表示，而数据库层的 CHECK 约束只有 user 和 admin 两种值，这样在联调时就会直接导致注册失败。修复后系统在接口层增加了字段映射，使得显示层语义和数据库约束保持一致。

### 2.2.2 视频上传

视频上传模块是整套系统真正的业务入口，也是最容易暴露性能问题的环节。系统目前支持的格式为 MP4、MOV、AVI、WebM，这些是视频网站比较常见的格式。在前端选择好视频并标注好动作后即可发起上传。由于后续还要执行抽帧、姿态推理、规则分析和模型调用，整个处理链路通常需要明显长于普通接口请求的时间，所以在这里不能采取同步阻塞的方式，否则前端会长时间无响应。

上传接口只做三件事，接收文件保存到安全目录、在数据库插入一条状态为 processing 的任务、返回唯一的任务编号给前端。真正的工作由后台的异步线程来分析，耗时比较长。前端再轮询单独跟踪任务的状态。接口可以尽快返回，前端页面不会长时间卡住，用户也能持续看到任务进展。

### 2.2.3 姿态多维特征提取

姿态特征提取是算法部分。系统最后是否有依据说断话，很大程度上取决于提供给系统的输入数据是否是稳定的、是否是有意义的。视频本身是一个片段连续像素的集合，并不是给人们提供直接打分的信息，算法系统需要先把原始画面转化成结构化的人体姿态数据。

具体来讲，这步做完至少有三层内容，第一层，抽帧、废帧过滤，尽量去掉准备动作、空画面、镜头变动阶段，第二层，解算颈、肩、肘、腕、髋、膝、踝等骨干节点三维坐标与置信度，作为下一步计算的根据地，第三步，在原始坐标基础上继续构造二级特征，比如大腿小腿夹角、左右骨盆的水平偏差，躯干倾斜程度等等，比起坐标，好对应到真实体式问题。

瑜伽评估并非判断某一帧是否“摆到了位”就可以，接下来要考虑动作维持阶段是否稳定，所以系统还对动作持续过程做了一些时序统计，比如角度波动、稳定性指标的计算，用来判断维持动作阶段的控制能力。

### 2.2.4 混合智能评估

评估报告最终是给用户直接阅读的，因此，系统没有只保留一个总分，而是沿用混合评估的思路，将本地规则系统和多模态模型结合起来使用。

其中规则系统要给出量化结果是什么，比如有哪些关节超出容差，哪些对齐关系不能达到标准，扣的分数是什么；多模态模型在结构化结果的基础上给出更加贴近我们日常表达的建议，将角度偏差和关键帧信息进行重新表达，可能使用更易懂的字眼。一方面我们可以保持可解释性，另一方面让结果更加可阅读。

### 2.2.5 结果展示

结果展示模块决定了用户能否切切实实真正地读懂评估结果，对于评估较为成功的任务，前端往往需要将后端返回的结构化数据转换成直观的界面，即分数区域，偏差列表，建议区域等，让使用者能较快地看出“整体表现如何、主要问题出在哪里、下一步该怎么改”。

异常情况同样需要认真处理，若评估流程中途失败，如视频内没有检测到有效人体、文件本身损坏，或者模型请求异常导致结果生成失败，后端不能只返回一个模糊的状态，需要把明确的错误原因也返回给前端，前端收到失败状态后会立即停止轮询，并展示错误提示，这样用户和开发者都可以更快的判断是文件本身、网络、还是服务本身的问题。

### 2.2.6 历史记录留存

系统还要有历史记录留存的特性，也就是将一次评估结果变成可持续跟踪的数据，对普通用户而言，系统要提供训练记录列表，可按时间倒序查看所有历史报告，点开每个报告后可比较不同阶段的练习表现情况；对管理员而言，系统要提供统计接口，可对平台的注册量、上传量、动作分布、平均分等进行统计，可从单条记录上升至平台运行视角。

## 2.3 系统性能

对于这样的大型计算项目而言，非功能性指标有时候更直接地决定了系统是否好用，甚至好用的体验比某个功能有没有更直接影响体验。对于一个系统，除了要满足功能需求以外，还要对性能以及稳定性有一定的要求。

#### （1）交互不能阻塞

文件上传、任务提交后的首次响应不能长时间等待，前端不能长时间等待无响应，所以系统必须将耗时计算与交互请求分离开来。

#### （2）轮询需要节流

前端查询任务进度应该采用固定的时间，如 3-5 秒，让用户可以感觉到变化，同时避免密集的请求给后端增加压力。

#### （3）外部服务需要容错

与千问模型通信需要超时和有限重试。外部的网络服务并不稳定，系统不能把主流程完全压在单次请求上。

#### （4）计算有兜底存储有兜底

如果出现非法文件、缺少动作参数、SQLite 锁冲突等问题，返回安全的降级结果，不会导致主进程直接崩溃。

#### （5）部署方案尽可能轻便

作为本科毕业设计的验证系统，本文尽量不依赖于 Redis、MySQL 主从等重型基础设施，而是把状态收敛到本地文件系统和 SQLite 中，保证普通的 PC 在完成配置后

即可运行。

这些指标虽然不代表功能模块那么直观，但是其实它们决定了系统是否能演示稳定，决定了系统能否扛得住联调，也决定了是否会让用户愿意继续使用。

在响应性方面，系统需要区分“接口响应时间”和“任务完成时间”。上传接口如果同步等待完整视频分析结束，接口响应时间就会被拉长到几十秒，用户体验很差；而采用异步任务后，接口只负责创建任务并返回编号，真正的任务完成时间通过轮询展示给用户。这样虽然总处理时间没有被缩短，但用户感知上的阻塞被显著降低。对于需要视频分析的系统来说，这种响应性设计比单纯优化某个函数执行时间更符合实际需求。

在稳定性方面，系统必须能够处理输入质量不确定的问题。用户上传的视频可能存在编码异常、时长过长、画面过暗、人物未完整入镜等情况。如果后端没有在文件校验和异常捕获阶段做好处理，这些输入很容易导致线程异常退出或任务状态无法更新。本文在实现中通过文件类型检查、异常捕获、任务状态回写和默认结果返回来增强稳定性，使系统即使不能完成理想分析，也能明确告诉用户失败原因。

在资源占用方面，视频分析天然比普通 Web 请求更重。抽帧数量如果过多，会增加 CPU 和内存压力；抽帧数量过少，又会影响动作分析完整性。系统最终采用限制最大处理帧数并保留关键运动帧的方式，在准确性和运行成本之间折中。对于本科毕业设计演示环境来说，这种折中是合理的，因为它既能避免机器长时间高负载，也能保证分析结果具有基本参考价值。

在可恢复性方面，系统需要保证失败不会扩散。模型接口失败时，应当退回规则评估；数据库短暂锁定时，应当通过等待和重试缓解；未知体式名称出现时，应当返回默认模板，而不是让空对象异常中断流程。本文把这些情况都纳入性能需求，是因为系统性能并不只体现在顺利情况下跑得快，更体现在异常情况下是否还能维持可解释的输出。

最后，系统性能还与论文结论的边界有关。本文没有声称系统已经完成大规模并发压测，也没有给出商业部署级别的性能指标。当前测试主要证明系统在本机单机和小规模任务下可运行、可反馈、可追踪。这样的表述更符合现有材料，也为后续工作留下了明确方向：如果未来继续扩展，就需要引入更严格的压力测试、队列调度和数据库迁移方案。



### 3 系统的设计

#### 3.1 系统体系结构

本系统在架构设计上遵循了软件工程中“关注点分离”的基本原则，将整体功能划分为四个相对独立、通过标准化接口进行通信的逻辑层次。这种分层设计并非预先设定的教条，而是在开发过程中，为应对混合型负载（I/O 密集型与 CPU 密集型并存）、多角色权限管理以及未来技术演进需求而逐步演化形成的合理结构。最终系统分层架构如图 3.1 所示。

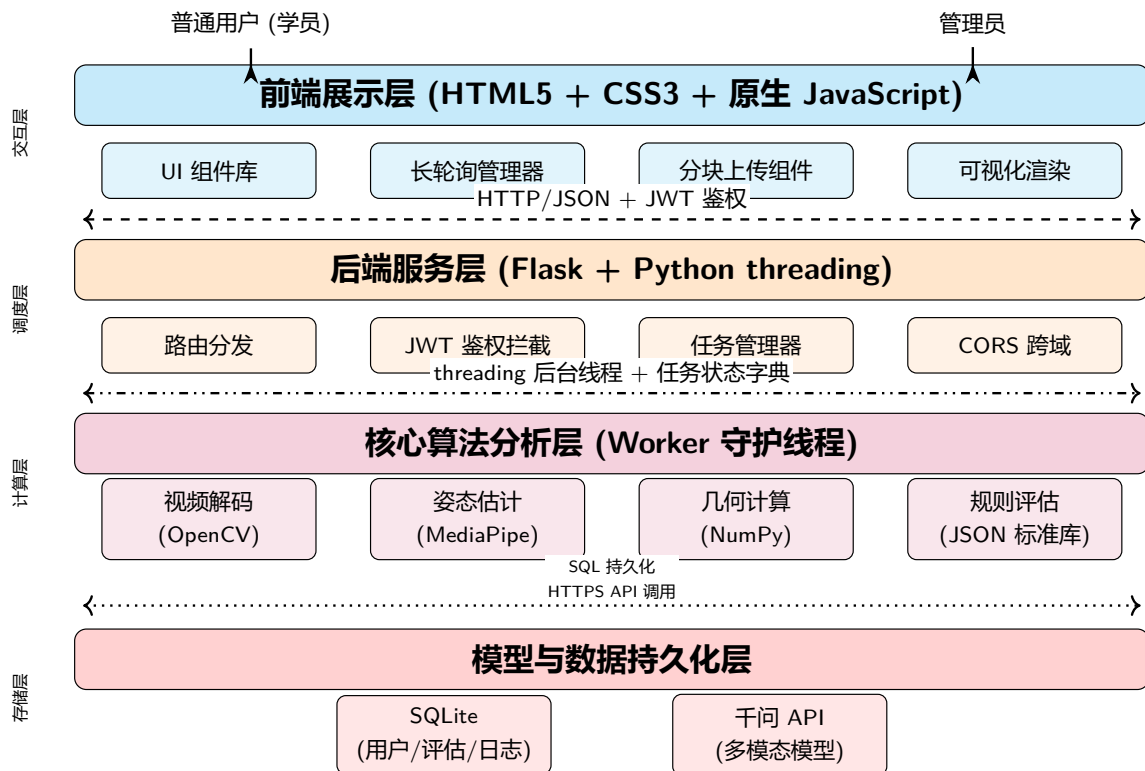


图 3.1 系统总体架构（前后端分离 + 后台任务调度）

##### 3.1.1 四层架构的职责界定

本节将对各层的职责划分进行详细阐述。

###### (1) 前端展示层：用户交互的聚合界面

前端展示层是直接面向最终用户的窗口，其职责不限于简单的界面渲染，而是涵盖了用户从发起请求到获得反馈的完整交互闭环。具体而言，该层主要包含以下功能模块：

###### (1) 鉴权表单

提供用户登录与注册的界面组件，收集用户凭证（如用户名、密码）并以安全方式（如 HTTPS POST 请求）提交至后端服务层。在成功获取 JWT 令牌后，前端需负责该

令牌在客户端侧的持久化存储（通常采用 `localStorage` 或 `sessionStorage`），并在后续所有需要身份认证的请求中自动将其附加于 `Authorization` 请求头中。

### （2）分块上传组件

针对用户上传的健身视频文件可能体积较大（如数十 MB 乃至数百 MB）且网络环境不稳定的实际情况，前端实现了文件分块上传机制。该组件将原始文件切割为若干较小数据块，依次向服务端发起上传请求，并支持断点续传与失败重试。这一设计显著提升了大文件传输的可靠性，并降低了单次请求对服务器内存的瞬时占用。

### （3）长轮询管理器

由于核心算法分析属于典型的异步、耗时操作（单次评估可能需要数秒至数十秒），前端无法通过同步 HTTP 请求等待结果返回。为此，该层封装了一个长轮询管理器。在向服务端发起分析请求并获得唯一的 `task_id` 后，管理器会周期性地向服务端查询该任务的执行状态。一旦状态变更为“已完成”，管理器即刻拉取评估结果并触发后续渲染流程。长轮询机制在实现难度与实时性之间取得了较好的平衡，是 Web 应用中处理异步任务状态的常见模式。

### （4）可视化渲染模块

在接收到后端返回的姿态评估结果（包含各关节关键点坐标、关节角度、规范性评分及文字建议）后，前端需将其直观地呈现给用户。具体实现上，该模块利用 HTML5 Canvas 或叠加于原生视频元素之上的图层，根据返回的骨骼点坐标数据，实时绘制人体骨架连线、关节角度标识，并以不同颜色（如绿色表示合格、红色表示需调整）高亮显示存在问题的身体部位。这种可视化反馈方式显著降低了用户对抽象数值的理解成本，提升了系统的可用性。

## （2）后端服务层：系统调度的逻辑中枢

后端服务层是基于 Flask 轻量级 Web 框架构建的，在整个系统中扮演着承上启下的枢纽角色。它不承担计算密集型任务，而是专注于请求的接纳、解析、分发与调度。其主要职责包括：

### （1）HTTP 路由分发与 CORS 配置

该层定义了完整的 RESTful API 端点集合，涵盖用户认证、文件上传、分析任务启动以及任务状态查询等核心业务接口。同时，为应对前后端分离架构下必然出现的跨域请求问题，服务端显式配置了 CORS 策略，通过在 HTTP 响应头中设置 `Access-Control-Allow-Origin` 等字段，告知浏览器允许来自指定前端域名的跨域访问。

### （2）JWT 校验与权限判断

对于所有需要身份认证的 API 端点，后端通过装饰器或中间件机制实现了统一的 JWT 拦截逻辑。每个请求到达业务处理函数之前，均会经过以下校验流程：① 从请求头 `Authorization` 字段中提取 JWT 令牌；② 使用服务端持有的密钥验证令牌签名，防止伪造或篡改；③ 检查令牌中的有效期声明（`exp`），拒绝已过期的令牌；④ 解析出载荷中的用户身份信息（如 `user_id` 和 `role`）。在此基础上，系统进一步实现了基于角色的

访问控制（RBAC）：普通学员仅可访问与自身相关的评估记录，而管理员则具备查询全量日志、管理用户等高级权限。

### （3）任务创建与后台线程调度

当接收到合法的姿态分析请求后，后端服务层不会在当前请求线程中直接执行算法（否则将导致 HTTP 响应长时间阻塞），而是执行以下调度逻辑：① 生成一个全局唯一的任务标识（`task_id`）；② 在内存中的任务状态字典（例如 `task_status = {}`）中初始化该任务的状态为“PENDING”；③ 调用 Python 标准库 `threading` 模块，启动一个独立的守护线程，该线程将承载后续所有计算密集型任务；④ 立即向前端返回 `task_id`，释放主线程以继续处理其他请求。这种异步任务调度模式是保证系统在多用户并发场景下仍能保持响应灵敏度的关键设计。

### （3）核心算法分析层：计算密集型的执行引擎

核心算法分析层是系统实现其业务价值的根本所在，所有与人体姿态评估相关的计算密集型任务均在此层完成。为避免阻塞后端服务层的主线程，该层的所有逻辑均运行于后台守护线程之中。其处理流程可分解为以下四个依次执行的阶段：

#### （1）视频解码（OpenCV）

后台线程首先获取用户上传视频文件在服务器上的存储路径，随后调用 OpenCV 库的 `VideoCapture` 类逐帧读取视频流。OpenCV 将视频中的每一帧解码为可供后续处理的图像矩阵（通常为 NumPy 数组格式，维度为高度 × 宽度 × 通道数）。该阶段需处理不同编码格式（如 H.264、MPEG-4）、不同分辨率和不同帧率的视频文件，对解码器的健壮性提出了要求。

#### （2）姿态特征提取（MediaPipe）

对每一帧图像，系统调用 Google 开源的 MediaPipe 框架中预训练的 Pose 模型进行推理。该模型能够在 CPU 上以较高效率检测并定位人体 33 个关键点（包括左肩、右膝、左脚踝等），每个关键点包含其二维图像坐标及三维世界坐标。相较于依赖专用 GPU 的深度学习方案，MediaPipe 在轻量化部署与实时性之间取得了良好的平衡，非常适合本系统的应用场景。

#### （3）几何计算（NumPy）

在获取各关节关键点的坐标数据之后，系统利用 NumPy 库进行高效的数值计算，以得到具有解剖学意义的姿态参数。以“膝关节角度”的计算为例，需要依次获取右髌、右膝、右踝三个关键点的坐标向量，计算膝-髌与膝-踝两个方向向量，然后通过向量点积公式  $\cos \theta = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}| |\vec{b}|}$  求得夹角弧度值，最后转换为度数输出。NumPy 的向量化运算能力确保了单帧中数十个关节角度的计算能够在毫秒级内完成。

#### （4）规则评估与评分

基于计算得到的各关节角度数据，系统依据预先定义的规则库对用户的动作规范性进行判断。例如，对于“深蹲”动作，规则库中可能包含以下判定逻辑：若膝关节角度介于  $120^\circ$  至  $150^\circ$  之间，则判定为“合格”；若小于  $120^\circ$ ，则判定为“屈膝过度”；若大

于 150°，则判定为“下蹲不足”。所有帧的评估结果将被汇总，生成最终的动作规范性得分、存在问题的帧索引列表以及相应的文字改进建议。

#### （4）模型与数据持久化层：存储与外部能力的接入点

该层承担着向内持久化存储与向外调用外部 AI 能力的双重职责，是系统内部数据流与外部服务交互的边界。

##### （1）向内存存储（SQLite）

本系统选择 SQLite 作为嵌入式关系型数据库，其无需独立部署服务器进程、单文件存储的特性使其非常适合桌面级或轻量级 Web 应用的后端存储需求。系统利用 SQLite 维护了多张数据表，主要包括：用户表（存储用户 ID、用户名、密码哈希值、角色权限等）、评估记录表（存储任务 ID、用户 ID、原始视频路径、评估结果 JSON、得分、创建时间等）以及系统日志表。这种持久化机制确保了用户数据在服务器重启后不会丢失，也为后续的数据分析与审计提供了基础。

##### （2）向外调用（千问 API）

单纯的规则评估输出（如“膝关节角度偏差 15 度”）虽然精确，但对于不具备专业知识的普通用户而言显得机械且不够友好。为增强评估结果的可读性与指导性，系统在获得规则评估的量化结果后，可选择性地构建一个多模态提示请求，调用千问大语言模型的 API 接口。例如，将“右膝角度 135 度，标准范围为 140–150 度”这一信息，连同用户当前姿态的关键帧图像（可选）一同发送至千问模型，请求其生成更自然、更具鼓励性质的动作改进建议。模型返回的文本内容随后被回写至评估记录表中，并最终呈现在前端界面上。

### 3.1.2 分层架构的工程化优势

上述四层分离的设计并非仅仅为了追求理论上的“整洁架构”，而是在实际开发与后续维护过程中，为解决真实问题而逐步形成的合理结构。具体而言，这种分层方式带来了以下两个可验证的工程化优势。

#### 优势一：职责边界清晰，显著提升故障定位效率

在软件系统的生命周期中，调试与排错往往占据大量时间成本。一个设计良好的架构应当能够帮助开发者快速缩小问题的可能范围。在本系统中，四层划分使得故障排查路径变得线性且可预期。

假设用户反馈“视频上传后长时间未获得评估结果”。在未分层的单体架构中，开发者可能需要在数据库查询、算法逻辑、前端请求代码之间反复跳跃，排查效率低下。而在本系统的分层架构下，可以按照以下顺序进行系统性的排查：

首先检查前端展示层——浏览器开发者工具中是否出现 JavaScript 错误？长轮询请求是否正常发出且得到响应？若前端无异常，则检查后端服务层——对应的 `task_id` 是否已在任务状态字典中生成？当前状态是“PENDING”、“RUNNING”还是“FAILED”？若状态为“RUNNING”但持续时间远超预期，则问题很可能位于核心算法分析层——

后台线程是否因异常而退出？OpenCV 是否在某帧解码时失败？MediaPipe 是否输出了空结果？若算法层日志一切正常，最后检查模型与数据持久化层——SQLite 数据库是否有写入权限问题？千问 API 调用是否因网络超时而未返回？通过这种逐层隔离、逐级检查的排查策略，系统故障的平均定位时间得到了显著缩短。

优势二：模块间低耦合，为技术演进与功能扩展提供良好支持

软件系统的一项基本矛盾在于：初始设计阶段的需求往往是不完备的，而系统上线后又必然面临功能迭代与技术更新的压力。分层架构的核心价值之一，就在于将变更的影响范围限制在某一特定层次内部，避免“牵一发而动全身”的重构风险。本系统的四层设计在这一方面体现得尤为明显。

具体而言，以下几种可预见的未来变更均可在不推翻整体架构的前提下完成：

#### （1）扩展动作库

若需要新增“俯卧撑”“硬拉”“引体向上”等评估动作，只需在核心算法分析层的规则评估模块中添加相应的判定逻辑与参考角度阈值。前端展示层（仅需在动作选择下拉框中增加新选项）、后端服务层（无需任何改动）以及持久化层均不受影响。

#### （2）替换姿态估计模型

若 MediaPipe 框架后续停止维护，或出现精度更高的替代方案（如 MoveNet 或自研模型），只需重写核心算法分析层中的“姿态特征提取”子模块，保持其输入（由 OpenCV 解码的图像帧）与输出（33 个人体关键点坐标）的数据契约不变，其余各层完全无感知。

#### （3）更换底层数据库

当业务规模增长，SQLite 无法满足高并发写入需求时，可将持久化层中的数据访问代码迁移至 MySQL 或 PostgreSQL。上层业务逻辑（如任务管理器中的状态字典操作、权限判断中的用户查询）由于不直接依赖具体数据库实现，只需修改数据访问接口的内部实现即可完成迁移。

#### （4）增加可视化形式

若未来需要提供“3D 骨骼动画”或“关节角度热力图”等更丰富的展示效果，变更仅需在前端展示层的可视化渲染模块内完成。后端服务层与算法层继续提供标准格式的关键点坐标数据，无需任何调整。

综上所述，四层分离架构不仅在系统实现阶段带来了清晰的开发者心智模型与便利的调试路径，更为系统的长期演化预留了充足的技术空间。这种设计在兼顾当前实现可行性（轻量级技术栈、快速迭代）与未来可扩展性（模块替换、功能追加）之间找到了一个合理的平衡点，是本系统得以顺利开发并具备持续演进能力的基石。

从实际开发顺序看，四层架构并不是一开始就完全定型的。早期原型更多关注算法是否能跑通，视频读取、关键点抽取和角度计算被写在较集中的脚本中，虽然调试方便，但很难直接支撑 Web 交互。后来尝试 Rust 后端时，系统在服务化方向上有了进一步探索，但与现有 Python 视觉生态、模型接口和前端调试节奏之间存在额外磨合成本。最

终采用 Flask 加原生 JavaScript 的方案，是在开发效率、依赖复杂度和答辩演示可控性之间做出的取舍。这个演进过程说明，架构选择并不是单纯追求技术新颖，而是要服务于系统目标和可落地性。

四层架构还让系统问题更容易定位。当前端页面无法显示结果时，可以先判断是展示层渲染问题、服务层接口问题、算法层分析问题，还是持久化层数据缺失问题。每一层都有相对清晰的输入输出，排查时不必在一大段混杂代码中寻找原因。开发中出现的角色字段冲突、任务轮询异常、数据库锁冲突和模型超时等问题，正是借助这种分层思路逐步定位并修复的。换言之，架构设计不仅影响最终代码组织，也影响开发者解决问题的路径。

同时，四层架构有助于把“算法能力”和“系统能力”区分开来。姿态估计算法可以提供关键点，但系统能力还包括用户管理、任务调度、异常兜底、结果保存和界面反馈。很多毕业设计容易把算法调用成功等同于系统完成，而本文的实现表明，算法只是中间环节之一。只有当算法输出能够被规则层消费、被模型层解释、被前端展示、被数据库留存时，它才真正转化为用户可使用的功能。

当然，当前架构仍然是面向本地演示和小规模使用的轻量设计。后台线程方式部署简单，但在大量并发任务下可能需要更专业的任务队列；SQLite 文件数据库便于复现，但面对高并发写入时会有天然边界；原生 JavaScript 前端依赖少，但当页面交互继续复杂化时，可能需要更系统的前端状态管理。本文保留这些边界说明，是为了让架构评价更加客观，也为未来工作提供清晰的演进方向。

### 3.2 系统运行环境配置

考虑到毕设演示和本地复现的需要，本文没有采用云原生容器或复杂分布式部署方案，而是搭建了一套本地可独立运行的环境。这样更适合答辩演示、联调和反复修改，也方便直接观察文件落盘、数据库写入、线程状态变化以及接口返回结果。

系统兼容 Windows 10/11，依赖 Python 3.10 及以上版本。底层视觉推理依赖 `mediapipe==0.10.7` 和 `opencv-python==4.8.1.78`。后端 Web 服务依赖 `Flask==2.3.3` 和 `flask-cors==4.0.0` 处理跨域访问。JWT 鉴权依赖 `PyJWT==2.8.0`，密码哈希使用 `bcrypt==4.0.1`。模型请求由 `requests==2.31.0` 发起。根据开发阶段的记录，项目前期还尝试过本地 Ollama + Qwen 的方案；而当前工程版本主要围绕千问多模态模型 API 做联调与集成。本地启动时，前端静态资源托管在 `localhost:3000`，后端接口绑定在 `localhost:5000`，两者通过标准 HTTP/JSON 通信，CORS 仅开放给指定端口。

这套配置依赖较少，也便于在开发机上直接观察文件、数据库和线程状态的变化。

从工程复现角度看，本地环境还有一个好处，就是每个中间产物都可以被直接检查。视频上传后的原始文件、抽帧后的关键帧、角度分析产生的 JSON、数据库中的任务状态以及前端轮询返回的响应体，都能够同一台机器上形成闭环。论文写作阶段之所以强调这一点，是因为本课题并不是单独验证某一个视觉算法，而是要证明一个包含用户

交互、后台调度、姿态计算、模型反馈和数据留存的系统能够连续运行。如果部署环境过早复杂化，问题定位反而会被容器、反向代理和云端权限等因素稀释，不利于毕业设计阶段对主要矛盾的呈现。

实际开发中，环境配置还直接影响了联调效率。比如前端静态页面和 Flask 后端分别运行在不同端口，能够较清楚地暴露跨域请求、鉴权头携带和接口路径拼接等问题；SQLite 以本地文件形式存在，也方便观察并发写入和事务提交是否符合预期；模型接口则通过独立配置项保存访问地址和密钥，便于在异常时快速切换到规则兜底流程。换言之，本文选择的运行环境并不是为了追求部署规模，而是为了让系统链路中的每一环都能被看见、被验证、被解释，这与本科毕业设计“可演示、可复现、可说明”的要求是相匹配的。

### 3.3 关键核心链路的技术实现

#### 3.3.1 非阻塞式媒体上传与守护线程分配

视频处理是整套系统最吃资源的部分，也是最容易引发前端卡顿和后端堵塞的环节，所以“防阻塞”不是附加优化，而是上传链路一开始就要考虑的前提。整个上传链路的实现细节如下：

**前端上传与进度反馈：**上传模块需要首先解决“用户是否知道系统正在工作”的问题。系统前端在提交视频后会立刻进入处理态，并在页面上显示上传和分析状态。这样做的原因很直接。视频评估链路明显长于普通接口请求，如果页面没有明确状态变化，用户很容易误以为系统已经卡死。在实现上，前端会保存任务编号，并依据状态接口返回内容持续更新进度文案和结果区域。

**后端文件校验与落盘：**当带视频文件的 `multipart/form-data` 请求到达后端时，系统先做基本合规校验，包括扩展名、文件大小和保存路径的规范化处理。上传成功后，文件会被写入固定目录，并记录到任务表中。开发中曾出现旧进程占用端口、旧后端未关闭和数据库路径混乱等问题，这些问题都说明文件和任务路径必须清晰统一，否则后续线程在定位视频或回写结果时很容易出错。

**任务创建与异步调度：**创建任务时，接口会生成全局唯一的任务编号，并将任务状态写入 `video_data` 表。最关键的一步，是让 Flask 主线程在完成入库后立即返回 HTTP 202 Accepted，而不是继续在当前请求内执行姿态识别和模型调用。真正耗时的分析工作由后台守护线程负责。这一改动来自联调中的真实问题。早期同步实现会让前端长时间等待，用户几乎看不到响应。改为异步后，上传接口能很快返回，前端再通过任务编号查询状态，系统交互体验明显改善。

**线程管理与状态同步：**后台工作对象会按顺序执行抽帧、姿态推理、规则评估和模型调用，并在每个阶段更新状态码供前端轮询。任务状态通过线程安全字典维护，必要时再同步到数据库。这样设计以后，前端不需要知道后台的每一段细节，只要根据状态码切换界面即可。对于开发者而言，这套状态流还可以帮助定位故障。例如任务若长期

停留在抽帧阶段，通常意味着视频读取有问题；如果长时间停在模型阶段，则更可能与网络或代理配置有关。

在实际联调中，状态码本身也方便排查问题。例如任务如果长期停留在“抽帧处理中”状态，通常说明视频读取阶段出了问题；如果长时间卡在大模型调用阶段，则更可能和网络或代理有关。上传链路采用分块和异步分离后，系统对网络波动的容忍度也更高。

为了避免后台线程在异常情况下悄悄退出，系统还需要在任务对象中保留错误信息和结束时间。这样做的意义并不只是在前端显示一段失败提示，更重要的是让一次失败能够被回溯。比如视频编码格式无法被 OpenCV 正常读取时，前端如果只收到“处理失败”，用户和开发者都很难判断问题来自文件本身还是后端程序；而当任务状态同时记录 `failed`、错误阶段和异常摘要时，问题定位就会清晰很多。本文在实现中将错误捕获放在后台任务最外层，保证即使抽帧、姿态推理或模型请求任一环节出错，任务状态也能被写回，而不是让前端一直停留在等待状态。

此外，异步上传链路还对用户体验产生了比较直接的影响。普通表单提交的等待通常只有几百毫秒到数秒，而视频分析任务可能持续几十秒。如果仍然采用同步阻塞方式，用户会无法判断系统是在工作、卡死还是请求已经丢失。改为“提交后立即返回任务编号，再由前端周期性查询”的方式后，等待过程被拆成了多个可解释状态，页面可以根据状态变化逐步展示“已上传”“正在抽帧”“正在分析”“正在生成反馈”等信息。虽然这些状态提示本身并不改变算法结果，却能显著降低用户的不确定感，也使系统更符合真实训练工具的交互逻辑。

### 3.3.2 基于 MediaPipe BlazePose 的空间特征提取

姿态解算是系统的数学基础，这一部分如果不稳定，后面的规则评分和大模型建议就很容易跟着失真。也正因为如此，本文在实现阶段花了不少时间做参数调优和误差观察，目标不是单纯追求每一帧都“看起来能识别”，而是让整段视频在常见居家拍摄条件下尽量保持可用状态。

视频抽帧策略：直接用 OpenCV 逐帧读取全部画面会造成很大的计算冗余，30fps 的 15 秒视频就有 450 帧，所以系统设计了一套自适应抽帧逻辑。系统先获取视频总帧数，再按“总帧数除以 60”的方式计算采样间隔，保证最多处理 60 帧。单纯均匀采样会漏掉动作关键帧，所以系统又加入了运动检测。它用帧差计算相邻帧差异，差异超过阈值（经验值 15%）的帧会被强制保留，即使不在采样点上。最终每个视频提取 40-80 帧不等，用来平衡精度和性能。

**MediaPipe 初始化与推理：**MediaPipe Pose 会把视频转成一系列离散的骨架点。系统在初始化时启用视频追踪模式，并设置基础置信度阈值，用来平衡识别稳定性和本地运行成本。MediaPipe 输出的并不只是二维像素位置，而是归一化的三维坐标和可见性信息。这一点对瑜伽场景很重要，因为系统不仅要看动作是否出现，还要进一步判断



关节角度、左右对齐和身体控制情况。



图 3.2 Ardhakati Chakrasana 姿态分析的关键帧示例 (frame\_4\_score\_0.98)

**坐标归一化与噪声过滤：**由于拍摄距离和画幅比例不同，直接使用像素坐标做角度计算会产生明显畸变，因此系统对关键点做了归一化处理，并重点保留肩、髋、膝和踝等与体式评价关系更强的部位。为了把离散点转成可解释的角度指标，系统会选取三个关键点形成两条向量，再用余弦关系计算夹角  $\theta$ ：

$$\theta = \arccos \left( \frac{\vec{A} \cdot \vec{B}}{|\vec{A}| |\vec{B}|} \right) \times \frac{180}{\pi} \quad (3.1)$$

在实际处理中，系统还会结合关键点置信度过滤低质量帧，并对连续角度值做平滑处理。这样可以减少单帧漂移带来的尖峰误差。居家拍摄时，光线变化和遮挡都很常见，因此这一处理对结果稳定性非常重要。

**时序特征提取：**单帧角度只能反映瞬时状态，无法说明动作维持阶段是否稳定，因此系统又设计了滑动窗口统计模块。系统会对连续帧的角度变化进行聚合，计算均值、波动幅度和稳定性特征。这些时序统计量随后会作为规则评估和大模型提示词的结构化输入。这样做的意义在于，瑜伽动作不是摆到某一刻就结束，维持过程本身也是动作质量的一部分。

这一步很关键。瑜伽动作不是摆到某个瞬间就结束了，练习者还需要维持、调整和

控制。系统把时间维度纳入分析后，结果会更接近真实训练反馈。

时序特征还能削弱单帧误差的影响。居家拍摄时，遮挡和光照变化很常见，单帧关键点偏移也会反复出现。系统引入窗口统计后，结论会更平滑，也更符合人工观察动作时的整体判断逻辑。

在关键点选择上，本文没有把 MediaPipe 输出的全部点都平均对待，而是依据瑜伽动作评估需求进行了侧重。面部、手指等细粒度关键点对整体体式判断的贡献相对有限，而肩、肘、髌、膝、踝以及脊柱方向更能反映动作是否稳定、是否对齐、是否存在代偿。因此系统在计算阶段优先保留这些骨架点，并围绕它们构造关节角度、左右侧差异和身体倾斜程度等指标。这样做可以减少无关关键点带来的噪声，也能让后续规则库更容易维护。

另一个需要说明的问题是二维视频对三维动作的表达限制。居家场景下通常只使用手机或电脑摄像头，拍摄角度往往固定在正面或斜前方。对于侧向弯曲、髌部旋转、肩部外展等动作，仅凭单视角视频很难完全恢复真实三维姿态。本文没有回避这一限制，而是在规则设计中更关注可观测的角度和对齐关系，并在反馈中避免给出超出观测能力的绝对判断。例如系统可以提示“当前画面中膝关节与髌部连线偏移较明显”，但不会把不可见方向上的细微旋转误判为确定错误。这种克制处理可以减少系统在复杂拍摄条件下的误导性输出。

从数据流角度看，姿态特征提取后的结果并不是直接生成最终评价，而是以结构化形式进入后续模块。每个任务会保留关键帧索引、主要关节角度、置信度、稳定性统计和规则命中项。这样做使系统具有较好的可解释性：当用户看到一个分数时，开发者可以继续追溯这个分数来自哪些关键点、哪些角度和哪些规则；当大模型生成的自然语言反馈不够理想时，也可以回到结构化输入检查是否存在特征缺失或提示词组织问题。由此可见，姿态提取模块既是算法计算层，也是整套系统可追溯性的基础。

### 3.3.3 构建规则专家评估系统

仅靠大模型这一类黑盒评估方式并不够可控，结果一致性也难以保证，因此系统专门构建了一套 JSON 驱动的本地动作标准规则库，用来承担基础评分和兜底任务。思路很直接，就是先把“哪些位置必须检查、偏差多大算问题、问题严重程度怎样映射到分数”固定下来，再让大模型在这个基础上做语言层面的增强，而不是反过来由模型决定全部结论。

知识库结构设计：规则库存放在 `config/pose_standards.json` 中。顶层以体式名称作为主键，每个体式下再定义目标角度、容差范围、关键对齐检查项和常见错误。这样设计有两个好处。一个是动作标准便于单独维护。另一个是新动作加入时不必修改大量业务代码，只要按既定格式补充规则即可。

阈值碰撞测试算法：规则评估器遍历上一步生成的时序角度矩阵数据，对每个关节执行：

```

deviation = abs(actual_angle - target_angle)
if deviation > tolerance:
    severity = min(deviation / tolerance - 1, 1.0) # 0-1 严重程度
    violations.append({
        "joint": joint_name,
        "actual": actual_angle,
        "target": target_angle,
        "deviation": deviation,
        "severity": severity
    })

```

每个违规项按严重程度加权扣分，最终综合得分  $S = 100 - \sum(w_i \times severity_i \times 10)$ ，其中  $w_i$  为关节权重（膝关节 0.3、脊柱 0.25、肩关节 0.2 等，总和为 1）。规则评估系统不仅提供了打分体系的量化底座，也是系统在没网环境下“柔性降级”的核心支柱——断网时直接返回规则评分和预设的通用纠正文案。

接口契约的摩擦与对齐：规则评估模块与其他模块对接时，系统真实遇到了前后端字段对不上的问题。前端统计视图依赖 `total_users` 字段渲染数据面板，但后端接口一度没有返回这一字段，导致页面异常。评估详情页还曾因为 `angle_data` 返回 `null` 而触发前端错误。针对这类问题，系统统一了接口响应封套，并约定缺失数据必须用空对象、空数组或默认值返回，而不是直接返回 `null`。统一响应结构如下：

```

{
  "code": 200,
  "data": {
    "angle_data": {}, // 即使无数据也返回空对象
    "suggestions": [], // 空数组而非 null
    "score": 0
  },
  "message": "success"
}

```

未检索到的数据块以空字符串 "" 或空集合 {}、[] 返回，严禁裸露 `null`。这个前后端共同达成的防御性约定，大幅削减了联调期间因字段缺失造成的无意义折腾。

这一约定主要解决联调阶段常见的字段缺失和类型不一致问题。即便模型接口暂时不可用，系统也还能给出基于角度和对齐关系的基础分析，不至于让整条业务链完全中断。每新增一个体式，都需要同步补充目标角度、容差范围和常见错误，规则库也据此持续维护。

规则库的存在还让评分过程具有相对稳定的边界。多模态模型的表达能力较强，但

它对同一组输入可能产生措辞差异，甚至在提示词约束不充分时补充一些系统并未验证过的判断。规则层则不同，它只根据已经计算出的角度、稳定性和对齐关系进行判断，输出结果更可控。本文将规则评估放在模型反馈之前，是为了确保系统先有一个确定的技术底座，再把自然语言生成作为增强环节。这样既能利用大模型的表达能力，又不至于把核心判断完全交给不可控输出。

在规则维护方面，系统采用“体式标准”和“错误解释”分离的思路。体式标准主要描述目标角度、容差范围、关键关节和权重；错误解释则描述偏差出现后应如何转化为用户能理解的反馈。例如膝关节角度偏差本身只是一个数值，但在用户侧需要说明为“膝盖伸展不足”或“支撑侧稳定性不够”。这种拆分能够降低后续扩展成本：如果新增体式，只需要补充标准；如果希望反馈更口语化或更学术化，则可以调整解释模板，而不必改动底层计算逻辑。

当然，规则方法也有明显边界。不同练习者的柔韧性、身体比例和训练阶段并不相同，同一个容差阈值不可能适用于所有人。因此本文没有把规则评分描述成绝对正确的判定，而是把它作为基础参考和异常兜底。对于本科毕业设计而言，这种处理更加务实：系统能够证明从视觉特征到结构化评价的链路是可行的，同时也保留了未来引入个体化阈值、自适应规则和更多体式样本的改进空间。

### 3.3.4 基于多模态大模型的认知转换与信息提取

为了把偏硬的骨架偏移数据转成更像真实教练会说的话，系统接入了千问多模态模型。这部分也是整个项目中调试周期最长的模块之一，因为它既涉及提示词设计，也依赖外部服务稳定性，还要处理模型输出格式不稳定的问题。任何一个环节出错，最终结果都可能“不像正常评价”，甚至无法被前端正确解析。

提示词工程与上下文构建：大语言模型本质上是文本生成器，默认更擅长输出连续叙述，但前端渲染需要的是得分、问题列表和建议条目等结构化结果。因此系统在发起请求前，对提示词做了更严格的约束。系统先构造 `system_prompt`，明确模型身份和输出格式；再构造 `user_prompt`，把体式名称、规则评估结果、时序特征和关键帧一起交给模型。这样做的目的，是尽量把模型输出限制在系统能够消费的范围。

#### 3.3.4.1 提示词工程与上下文构建实现

系统提示词的设计：系统构造的 `system_prompt` 是一个明确的角色和格式定义，它告诉模型“你是谁，你该做什么，你应该怎样输出”。一个典型的系统提示词示例如下：

你是一位专业的瑜伽教练。你需要根据学员的动作视频分析结果，提供结构化的评价反馈。你的输出必须严格遵循 JSON 格式，包含以下字段：

```
{
  "overall_score": <0-100的整数，代表整体动作规范度>，
```

```

"assessment_level": <"excellent"/"good"/"fair"/"needs_improvement">,
"key_issues": [
  {
    "joint": <关节名称>,
    "problem": <具体问题描述>,
    "severity": <"high"/"medium"/"low">
  }
],
"suggestions": [
  <针对每个问题的改进建议，保持自然语言，鼓励性表达>
],
"positive_feedback": <对做得好的地方的鼓励>
}

```

重要：必须返回有效的 JSON，不要有前后额外文本。

这样的 `system_prompt` 不仅声明了模型的角色（教练），还通过 JSON Schema 示例明确了预期的输出结构。这种“架构优先”的设计相比于自然语言要求更具约束力。

用户提示词的上下文构建：系统接下来构造 `user_prompt`，这是将实际的分析数据和视频图像组织成模型能理解的、有逻辑的信息包。一个典型的用户提示词构建流程如下：

用户提示词 = f"""

动作分析任务：{pose\_name} ({difficulty\_level})

#### 【规则评估结果】

总得分：{rule\_based\_score}/100

评估时间：{duration}秒

关键角度数据（时序均值 ± 标准差）：

- 膝关节：{knee\_angle:.1f}° ± {knee\_std:.1f}°  
目标范围：140-150°，评估：{'合格' if knee\_valid else '不合格'}
- 髋关节：{hip\_angle:.1f}° ± {hip\_std:.1f}°  
目标范围：90-100°，评估：{'合格' if hip\_valid else '不合格'}
- 脊柱伸展：{spine\_extension:.1f}° ± {spine\_std:.1f}°  
目标范围：15-25°，评估：{'合格' if spine\_valid else '不合格'}

### 【问题总结】

- `{len(violations)}`处偏差
- 最严重的问题: `{top_violation}`
- 稳定性评分: `{stability_score:.0f}%`

### 【关键帧图像】

[已包含3张示意图，展示起始、中间、结束姿态]

请结合上述量化数据和视频内容，给出综合评价。

"""

这种构建方式的核心特点在于分层递进：先给出量化的规则评估作为“硬参考”，再提供时序统计削弱单帧噪声，最后才是视觉化的图像材料。这样模型输出时既有量化依据，也有定性理由。

多模态内容的组织：用户提示词中的图像不是简单地追加，而是精心选择的关键帧。系统会从整个视频中抽取起始、中间和结束三个代表性帧，按照：

#### （1）起始帧

展示学员进入体式前的预备姿态

#### （2）中间帧

展示维持体式时最稳定的一帧（置信度最高、角度最接近目标的帧）

#### （3）结束帧

展示学员离开体式或姿态开始松散的状态

这三帧与上文的规则评估数据形成了“文本-视觉”对偶。当模型既看到了硬指标（“膝关节 145°”）又看到了对应的视觉（膝部弯曲的照片），其输出的准确性和可解释性会明显提高。

**API 调用与容错机制：**千问 API 的调用封装在 `QwenClient` 类中。核心方法 `analyze()` 执行以下流程：

#### （1）构造 `messages` 数组

包含 `system` 和 `user` 两条消息；`user` 消息的 `content` 使用“文本提示 + 多张 `image_url` 图片”的多模态数组结构。

#### （2）设置超时与重试机制

设置 `timeout=30` 秒，`max_retries=2`，重试间隔 2 秒。

#### （3）调用请求接口

若超时或连接错误，则触发重试；两次失败后抛出自定义模型调用异常。

#### （4）异常处理

上层调用方捕获异常后，激活规则降级补偿策略，返回纯规则评估结果。

响应解析与数据提纯：大模型返回的 `response.choices[0].message.content` 实际上仍是字符串。即使提示词已经要求按 JSON 返回，模型偶尔还是会在前后附带额外说明，或者轻微改动字段名，因此系统又设计了一套解析和补救流程：

（1）JSON 提取与解析

用正则 `r'{[\s\S]*}'` 提取第一个完整 JSON 对象。

（2）解析失败处理

尝试 `json.loads()`，若失败则尝试用 `ast.literal_eval()`。

（3）若仍失败，遍历常见错误模式

中文逗号替换为英文逗号、单引号替换为双引号、缺失引号的 key 补全。

（4）解析成功后，执行字段存在性校验

缺少建议列表则补空列表；总分字段不是数字时退回规则打分；字段类型不匹配时再做强制转换。

这一系列处理的核心目的，是避免模型输出格式不稳定时直接冲击前端展示层。

隐形代理劫持导致千问请求阻断：在测试阶段，系统遇到过一个很典型但不容易马上发现的问题。账号凭证正常，单独使用 Postman 可以请求成功，但一旦在 Flask 主程序里发起相同请求，调用就会长时间挂起直至超时。后来通过检查系统环境变量，确认开发环境中残留了代理相关配置。底层 `requests` 库会默认继承这些代理设置，从而把请求导向本地代理端口，最终导致模型服务不可达。项目最后在应用入口主动清理相关环境变量：

```
os.environ.pop('HTTP_PROXY', None)
os.environ.pop('HTTPS_PROXY', None)
os.environ.pop('ALL_PROXY', None)
```

成功打通认知引擎传输大动脉。

这一问题说明，多模态模块的不稳定并不一定来自模型本身，也可能来自运行环境和代理配置。提示词工程也经历了多次调整。最终系统采用“结构严格、文本适度自然”的策略，也就是要求模型按 JSON 返回关键字段，同时允许建议部分保持自然语言表达。

在模型反馈内容的控制上，本文还特别关注了“建议是否可执行”这一点。早期模型输出虽然语气自然，但有时会停留在“保持稳定”“注意发力”这类较宽泛的表述，用户看完以后仍然不知道下一次练习该怎么调整。为此，系统在提示词中增加了问题部位、偏差方向和改进动作三类信息，要求建议尽量对应到具体关节或身体区域。例如当规则层识别出膝关节伸展不足时，提示词会同时传入目标角度、当前角度和偏差程度，模型再把这些信息转化为“练习时先稳定支撑脚，再逐步打开髋部和延展膝关节”这样的反馈。这样生成的建议既保留自然语言可读性，又不会脱离结构化事实。

为了减少模型“过度发挥”，系统在输出校验阶段还会把模型返回内容与规则结果

做一次简单一致性检查。如果模型指出的问题关节完全不在规则命中列表中，系统不会直接把它作为核心问题展示，而是降级放入补充说明或直接忽略。这样处理的原因是，当前项目的视觉输入和样本规模有限，模型可能根据画面做出一些带有主观推断的解释；而论文所需要证明的是一条可靠的工程链路，不能把未经规则或数据支持的结论写入最终报告。通过这种“规则先行、模型辅助”的方式，系统输出在可读性和可控性之间取得了比较合适的平衡。

从论文论证角度看，多模态模块的价值并不只是让反馈文字变得更像教练，而是让结构化数据能够进入用户可理解的语义层。角度、方差、置信度和偏移量对于开发者有意义，但普通练习者更关心“哪里不对”和“怎么改”。系统通过模型把这些数据转译成建议文本，使计算结果具备了实际使用价值。与此同时，模型模块被限定在解释和表达层，不直接篡改规则评分，这也避免了系统核心判断被外部服务波动完全左右。

### 3.3.5 数据库争用锁死与读写安全区隔离

SQLite 在单文件并发读写场景下暴露过明显问题。系统支持后台异步任务独立运行，而这些任务会持续写入任务状态和分析结果；同时前端还会周期性发起轮询请求读取状态。读写混合后，系统一度频繁出现 database is locked 错误，甚至伴随 journal 文件残留。这个问题直接影响了评估流程的稳定性，也是联调中最需要优先解决的基础问题之一。

修复这个瓶颈主要分为两步。第一步是调整数据库文件的存放方式，避免把数据库放在频繁热更新或监控的工作区。第二步是在连接建立时启用更适合并发读写的 WAL 模式，并设置合理的等待超时。这样以后，读请求与写请求之间的冲突显著减少，系统在并发任务和轮询并存的情况下更加稳定。关于更高并发下的吞吐极限，本文没有做完整压力基准，因此这里只能说明修复后系统在当前测试规模下已明显稳定，不能进一步外推更大规模结论。

### 3.3.6 作用域逃逸引发的数据库触发器失效

数据库除了存数据，还承担了部分自动汇总逻辑。系统原本设计在评估记录表插入一条新记录后，通过触发器自动更新用户训练进度表。

但旧版 SQL 初始化脚本里，触发器内部计算均值时错误引用了全局模糊字段域：

```
CREATE TRIGGER trigger_update_user_progress
AFTER INSERT ON assessment_record
BEGIN
    UPDATE user_progress
    SET total_sessions = total_sessions + 1,
        avg_score =
            (avg_score * total_sessions + NEW.score)
            / (total_sessions + 1)
```



```
WHERE user_id = user_id; -- 这里的 user_id 引用模糊!
END;
```

由于没用 NEW 关键字对旧事务行精准绑定，WHERE user\_id = user\_id 实际上永远为真，导致更新了所有用户的进度记录。事务落盘前最后校验阶段，数据库引擎报 no such column: user\_id 异常（因为触发器内部把 user\_id 既当列名又当变量）。由于发生在事务内部，整个评估入库操作被撤销回滚。

经过逐步追踪 SQL 执行过程，问题最终定位到触发器内部的列引用方式。系统随后删除旧触发器，并用明确的别名约束重新创建：

```
CREATE TRIGGER trigger_update_user_progress
AFTER INSERT ON assessment_record
BEGIN
    UPDATE user_progress
    SET total_sessions = total_sessions + 1,
        avg_score =
            (avg_score * total_sessions + NEW.score)
            / (total_sessions + 1)
    WHERE user_id = NEW.user_id; -- 明确使用 NEW 别名
END;
```

这一修改直接修复了评估记录入库后进度表更新失败的问题，也解释了前期“评估失败但主流程看似正常”的现象来源。

### 3.3.7 空值对象引用引发的雪崩

系统在处理未收录体式名称时，还出现过一次典型的空对象异常。用户输入知识库未覆盖的动作名后，后端在读取规则配置时得到 None，随后下游逻辑直接对其做集合判断，最终抛出 TypeError: argument of type 'NoneType' is not iterable。

栈追溯发现，从动作配置字典获取特征库时：

```
pose_standards = POSE_LIBRARY.get(pose_name) # 返回 None
if "common_errors" in pose_standards: # 对 None 做 in 操作，崩溃
    ...
```

下游规则评估模块如果不先做空值检查，就会把这一错误继续放大。修复后，系统增加了默认模板和降级逻辑：

```
pose_standards = POSE_LIBRARY.get(pose_name)
if pose_standards is None:
    pose_standards = DEFAULT_FALLBACK_STANDARDS # 返回通用空模板
    logger.warning(f"Unknown pose '{pose_name}', using fallback")
```

所有未命中体式都会被转成空模板或走降级流程。这样一来，即使输入不在动作标准库里，主流程也不会直接崩溃。

### 3.3.8 前端核心模块实现与优化

前端虽然不直接参与算法计算，但交互复杂度并不低，几个关键模块的实现和联调经验仍然值得记录。

**长轮询状态管理器：**前端在拿到任务编号后会启动轮询管理器。它定时请求状态接口，并根据 `status` 字段切换界面。任务为 `processing` 时显示处理提示，任务为 `completed` 时停止轮询并渲染结果，任务为 `failed` 时展示失败原因。为避免重复启动轮询，前端还增加了防抖控制。

**结果可视化渲染：**评估结果本质上是一个嵌套 JSON，其中包含分数、角度偏差和建议列表。前端需要把这些结构化数据转换成可阅读界面。项目联调中发现，只要后端字段命名稍有不一致，前端展示就会立即出错，因此结果渲染不仅是界面问题，也是接口契约是否稳定的直接体现。

**上传进度与恢复：**大视频上传容易因网络波动失败，因此前端在交互设计上保留了进度提示和恢复思路。即便当前系统规模不大，这一设计仍有实际意义，因为评估任务往往伴随明显等待时间，用户比普通表单提交更需要明确的过程反馈。

### 3.3.9 视频处理与内存管理优化

长时间运行的视频处理任务容易带来内存占用累积问题。为减少资源长期堆积，系统在视频处理完成后会主动释放 OpenCV 与 MediaPipe 相关资源，并在必要时触发垃圾回收。当前项目做过连续处理场景下的运行观察，未出现明显的持续增长趋势，但本文没有给出更严格的内存基准测试数据，因此这里只能说明系统在当前测试规模下能够保持基本稳定。

## 3.4 数据库设计细节

### 3.4.1 持久化存储设计原则

基于免安装、本地演示和便于答辩环境复现的核心诉求，系统没有继续引入部署成本更高的 MySQL 或 PostgreSQL，而是选用单文件、零配置的 SQLite 作为数据底座。这个选择并不意味着数据库设计可以随意简化，相反，正因为 SQLite 在并发写入场景下更容易暴露问题，所以表结构、索引策略和读写方式都需要提前考虑。

数据库建模遵循业务隔离原则，尽量降低表间硬连接，避免大表 Join 造成性能劣化。所有表均设置自增整型主键，外键约束只做显式声明而未强制启用。由于 SQLite 默认关闭外键，需要手动设置外键开关，这样既保留了引用关系的设计意图，也在一定程度上减少了本地演示环境里的锁开销。

3.4.2 数据库实体与核心关系拓扑

系统中流转的核心数据实体被拆分为五张相互协作的表。表结构映射关系如图 3.3 所示。

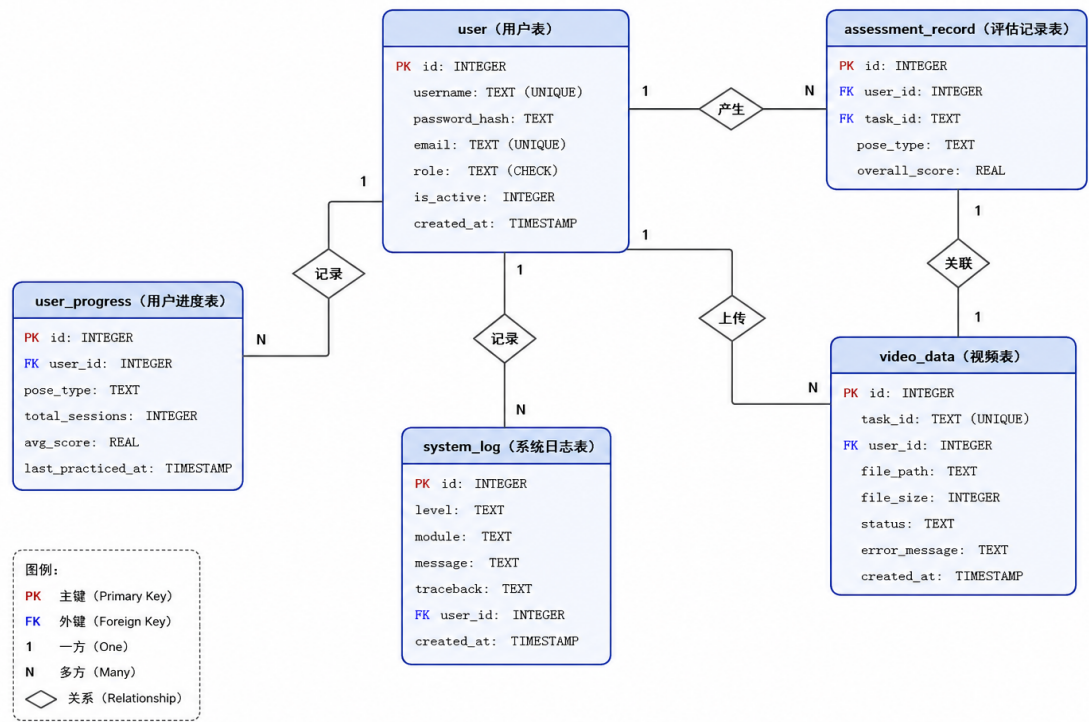


图 3.3 系统底层关系型实体流转 ER 图

核心表结构说明如表 3.1 所示。

表 3.1 核心数据表物理模型结构说明

| 实体表名              | 业务逻辑与说明                                                                           |
|-------------------|-----------------------------------------------------------------------------------|
| user              | 系统最高级身份凭证底座，字段包括用户编号、用户名、密码哈希、邮箱、角色、启用状态和创建时间。其中用户名和邮箱需要保持唯一，角色字段通过约束限制为普通用户或管理员。 |
| assessment_record | 系统价值核心表。承载每次体式评级，字段包括记录编号、用户编号、任务编号、体式类型、总分、角度数据、问题列表、建议列表、模型原始响应和创建时间。           |

| 实体表名          | 业务逻辑与说明                                                                  |
|---------------|--------------------------------------------------------------------------|
| video_data    | 文件系统抽象映射层。字段包括视频编号、任务编号、用户编号、文件路径、文件大小、处理状态、错误信息、创建时间和完成时间。              |
| user_progress | 个人健身履历表。字段包括进度编号、用户编号、体式类型、训练次数、累计平均分和最近训练时间，由数据库底层触发器维护一致性。             |
| system_log    | 安全与维保探针。字段包括日志编号、日志级别、来源模块、日志内容、异常栈、关联用户和创建时间，用于记录系统全生命周期内的错误、告警和越权监控信息。 |

这五张表并不是简单并列关系，而是分别承担不同职责。用户表负责身份管理，视频表负责关联文件和任务状态，评估记录表负责保存评估结果，用户进度表负责用户维度的阶段汇总，系统日志表负责记录异常与运维痕迹。这样拆分以后，系统在查询、追踪和排错时会更清楚，也更不容易把不同用途的数据混在同一张表中。

这套实体关系设计也服务于两个常见需求。普通用户查看历史训练记录时，可以从用户维度直接追到报告和视频任务；管理员做平台统计时，可以把单次任务结果进一步汇总到用户进度和系统日志层面。数据库设计不只是为了存储数据，也需要为后续查询、分析和排错预留路径。

### 3.5 系统控制流与时序设计

为了让异步交互保持可控，系统在实现时把关键业务流程整理成时序设计。这样做是为了先理清“谁发起请求、谁负责校验、谁返回状态、谁更新结果”这些关系，减少前后端理解不一致和线程状态难以追踪的问题。核心场景如下。

用户注册认证时序（图 3.4）：

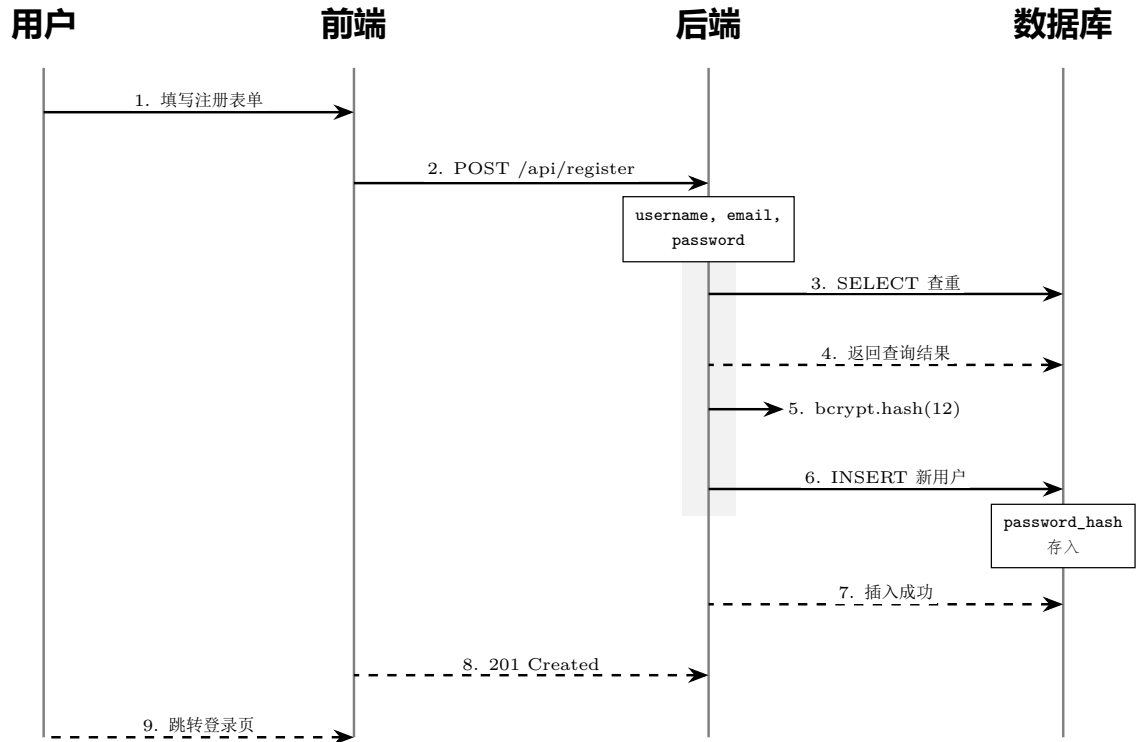


图 3.4 用户注册时序（密码防重 + 哈希存储）

用户注册认证时序完整展示了系统从接收用户注册请求到完成账号创建的全过程。首先，用户在前端页面填写用户名、邮箱以及密码等注册信息，前端在完成基础格式校验后，通过 `POST /api/register` 接口将数据发送至后端服务。后端接收到请求后，并不会直接写入数据库，而是优先执行用户名与邮箱的唯一性检查，通过 `SELECT` 查询判断当前账号信息是否已经存在，以避免重复注册问题。在确认用户信息不存在后，系统进入密码安全处理阶段。后端利用 `bcrypt.hash(12)` 对用户输入的明文密码进行加密处理，其中参数 12 表示哈希计算的复杂度等级。采用 `bcrypt` 算法能够有效防止彩虹表攻击和数据库泄露后的密码逆向破解，提高系统整体的账户安全性。加密完成后，系统仅将生成的 `password_hash` 保存到数据库中，而不会直接存储用户原始密码，从根本上降低了敏感信息泄露风险。随后，后端执行 `INSERT` 操作，将新用户信息写入数据库。当数据库返回插入成功结果后，后端向前端返回 `201 Created` 状态码，表示用户账号创建成功。前端接收到响应后，再执行页面跳转或提示注册成功等操作，完成整个注册流程。该时序图不仅体现了前后端分离架构下的接口交互逻辑，也强调了用户认证过程中“先校验、后加密、再存储”的安全处理顺序。如果忽略用户名查重、密码哈希或状态码规范返回等步骤，系统将容易出现账户冲突、密码泄露以及接口安全性不足等问题。因此，在实际开发中，注册认证流程必须严格按照该时序执行，以保证系统的安全性、稳定性与数据一致性。

用户登录与 JWT 签发时序（图 3.5）：

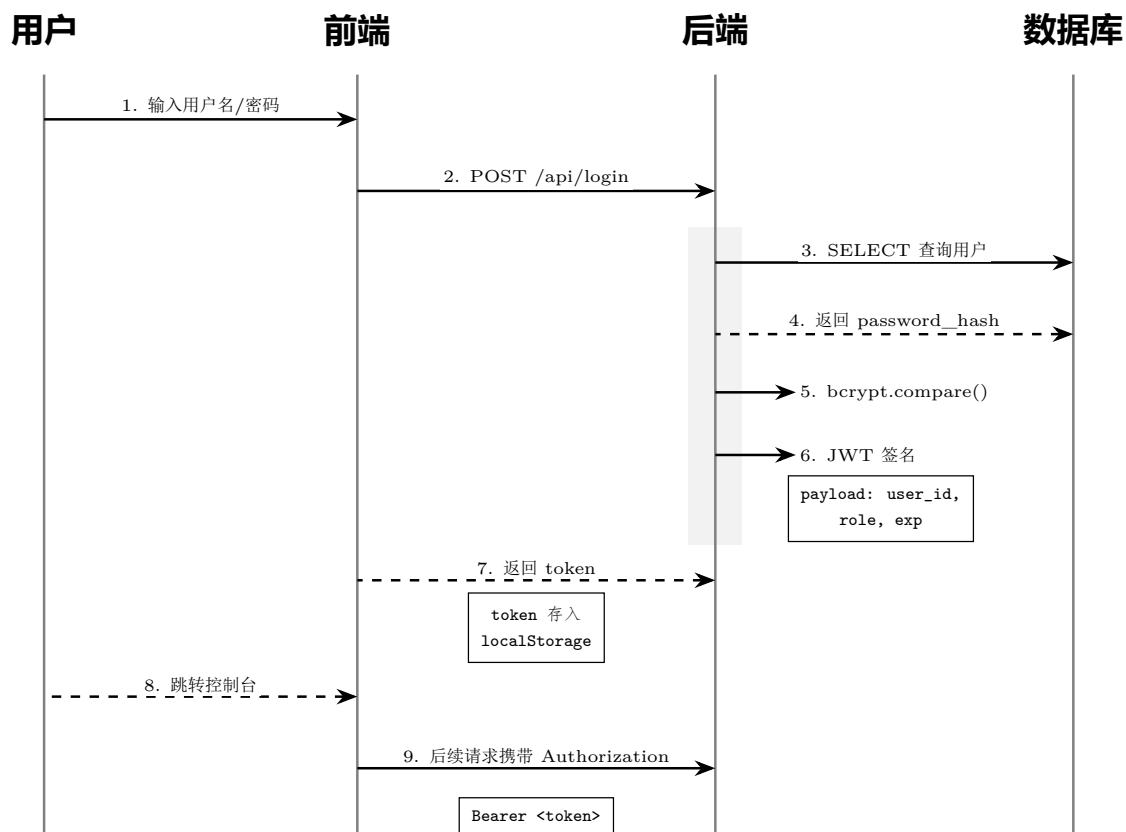


图 3.5 无状态 JWT 会话校验与登录时序

登录时序图体现了无状态鉴权的实现思路。服务端不长期保存会话上下文，而是通过 JWT 封装用户身份和生命周期信息。这样能简化后端状态管理，但也要求前端在后续请求中正确携带和处理凭证。

高并发评估调度时序（图 3.6）：

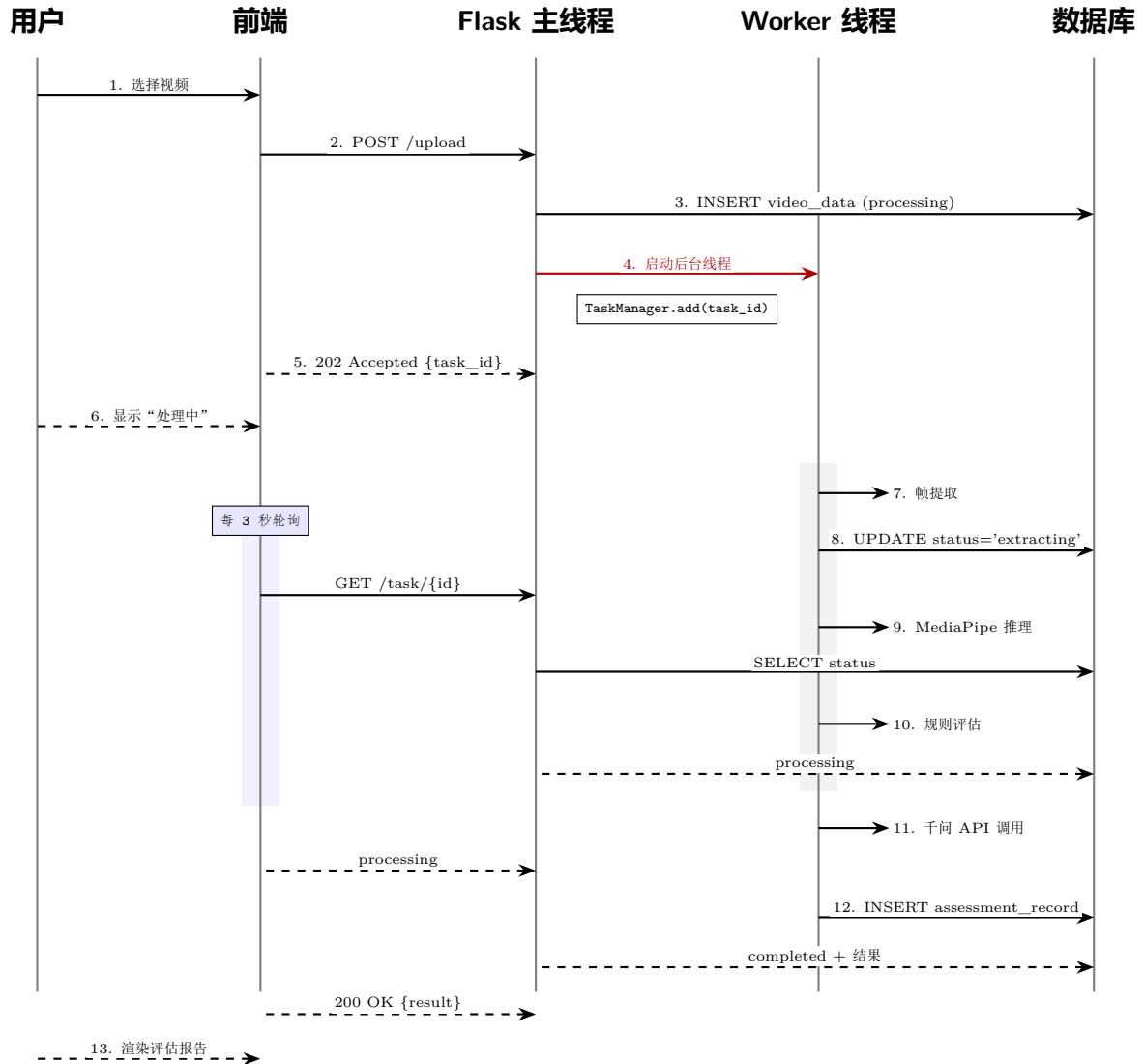


图 3.6 视频异步评估调度时序（含轮询闭环）

这一部分直接决定用户等待过程是否平滑。用户看到的只是“上传后正在处理中”，但背后还经历了任务入队、线程启动、状态更新、轮询查询和结果回写等多个步骤。只有这些步骤协同顺畅，前端才能稳定呈现进度和结果。

管理员防越权干预时序（图 3.7）：

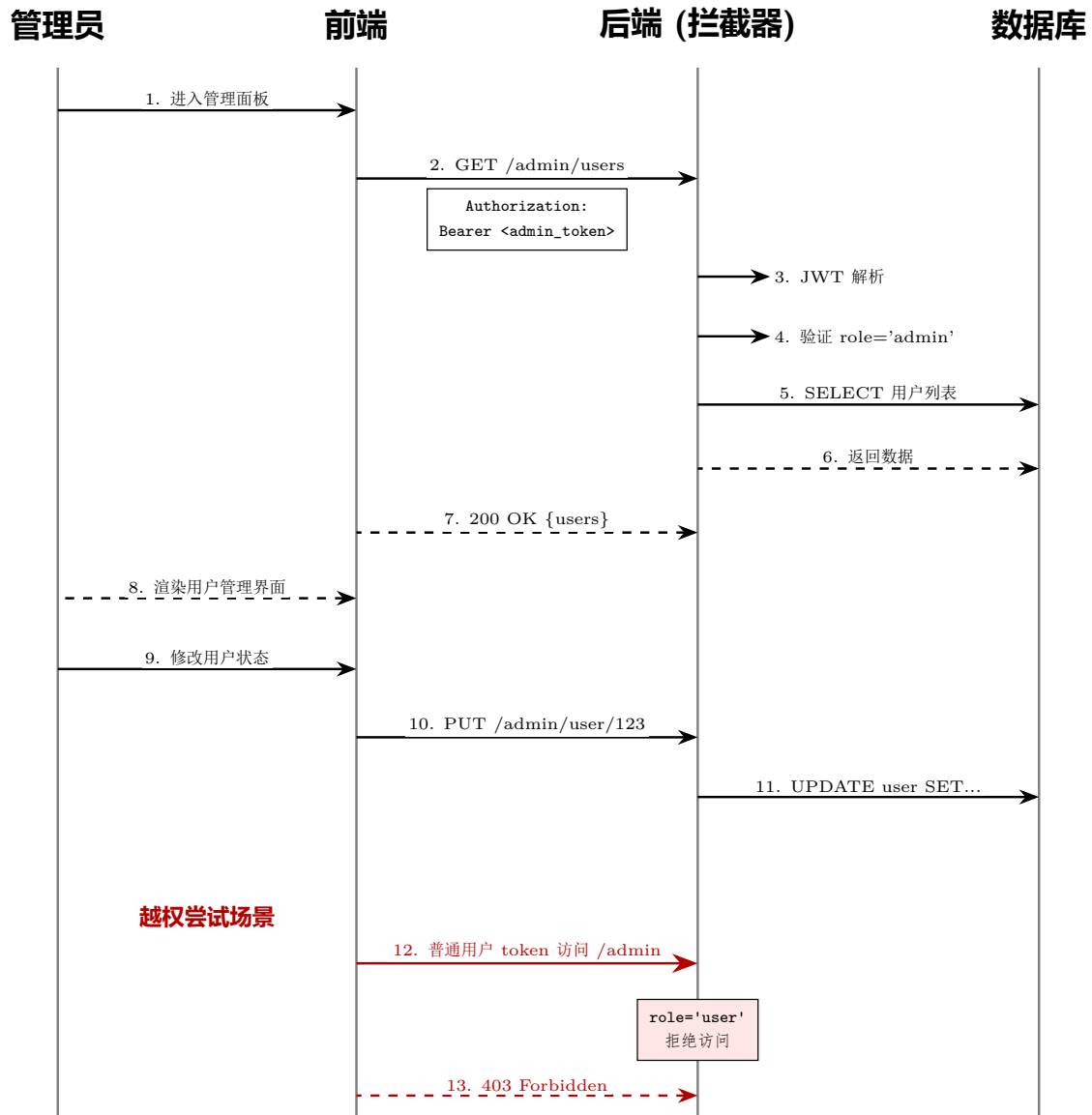


图 3.7 管理员提权操作时序（基于角色路由校验）

管理员时序设计的意义，在于把“能做什么”和“谁能做”明确分开。普通用户请求即使进入相同路由前缀，只要角色不满足条件，也必须在拦截层直接被拒绝，这样才能避免越权修改用户状态的风险。



## 4 系统测试

### 4.1 测试环境与总体测试策略

为了让测试结果尽量接近最终演示和实际使用状态，本文没有把验证工作局限在单一函数级别，而是把多数测试放在接近实际运行环境的本地 Windows 物理机上完成。很多问题只有在真实的视频读写、数据库访问、线程切换和接口调用同时发生时才会暴露出来，因此仅靠局部单元测试并不足以说明整个系统是否可靠。

本文采用的测试策略没有完全沿着“代码覆盖率优先”的路线展开，而是更强调“全链路集成测试”与“异常注入测试”的结合。前者主要验证从前端提交到后端处理再到结果返回的闭环是否稳定。后者则主动制造弱网、异常输入、角色越权和依赖故障等场景，观察系统能否给出合理响应，而不是直接失控。

测试环境与开发环境保持基本一致，目的是减少“开发时能跑、演示时不能跑”的情况。前端通过本地浏览器访问静态页面，后端以 Flask 服务方式启动，数据库使用同一份 SQLite 文件，模型调用使用实际配置的外部接口。在测试过程中，系统会保留任务编号、后端日志、数据库记录和关键帧输出，这些材料可以相互印证。例如前端显示任务已完成时，数据库中应当存在对应的 `assessment_record`；如果前端显示失败，日志中也应能找到失败阶段和异常摘要。这样的测试方式虽然不像大型工程那样完整自动化，但更符合当前课题规模，也能较有效地覆盖毕业设计答辩时最可能被问到的系统可用性问题。

此外，本文在测试中并没有只选择最理想的视频样本，而是保留了开发过程中出现过问题的样本和异常场景。这样做可以避免测试结论过分依赖一次顺利演示。对于一个以居家训练为目标的系统来说，用户上传的视频质量天然不稳定，可能出现光照不足、人物位置偏移、视频编码不兼容、体式名称输入不规范等情况。如果测试只验证标准样本，系统看起来会更完整，但结论的说服力反而不足。因此本文把异常输入和故障复现放入测试章节，是为了更真实地呈现系统已经解决的问题和仍然存在的边界。

### 4.2 核心功能集成测试的开展与剖析

核心功能集成测试主要检查模块串联后的实际表现。很多问题只有在前端请求、后端接口、数据库状态和异步线程同时联动时才会出现，因此这一部分是验证系统是否真正可用的关键。

#### 4.2.1 安全模块与角色约束链测试

这项测试主要验证系统的安全边界是否真正实现到位。用常规注册流程进行联调后，最早暴露的问题出现在角色映射层。前端为了页面显示友好，提交了 `role='learner'` 字段；后端最初没有做统一过滤，请求直接进入 SQLite。数据库层由于 CHECK 约束只允许 `'user'` 和 `'admin'` 两个值，因此注册流程直接失败。

后续修复中，系统把默认入库角色统一重置为 'user'，并在返回前端展示时再映射成 'learner'。这组测试说明，前端可以保留更友好的业务语义，但数据库层必须保持严格约束，二者之间需要通过明确映射连接起来。

#### 4.2.2 非阻塞上传与心跳轮询调度测试

这项测试验证系统在长视频场景下是否仍能保持前端可交互。根据实际联调记录，上传接口在异步改造后能够先返回 202 状态和任务编号，前端据此进入轮询。后端则在后台线程中继续执行抽帧、姿态分析、规则计算和模型调用。测试过程中，页面能够持续显示处理中状态，并在任务完成后自动切换到结果展示。与早期同步实现相比，用户等待体验明显改善。

这组测试说明，在当前系统规模下，“接口快速返回 + 后台线程异步处理 + 固定间隔轮询”的组合可以正常工作。前端没有出现长时间假死，后端也没有因为单个视频任务阻塞后续请求。至于更大规模并发的极限性能，本文没有做严格压测，因此不作进一步扩展结论。

#### 4.2.3 极限状态下的多模态认知链路试错测试

为验证大模型评估链路的稳定性，本文还构建了异常环境测试。正常情况下，模型能够结合关键帧和结构化指标返回较完整的评价结果。切换到网络异常或代理干扰场景后，后台会出现 `urllib3.exceptions.MaxRetryError` 或 `requests.exceptions.ConnectionError` 等典型错误。

外层调度器在捕捉到超时或连接异常后，没有让任务整体失控，而是激活规则降级策略，调用本地动作标准完成基础评估并把结果返回给前端。这样一来，即使模型服务暂时不可达，系统仍能保留一条基本可用的评估路径。

### 4.3 研发踩坑与真实故障根因重现分析

项目开发过程中，很多问题并不是写代码时立刻就能预判到的，而是在系统真正跑起来以后才暴露出来。论文把故障分析单独列出，主要是为了说明这些问题的出现方式、定位过程和修复结果，也让后续章节中的设计取舍有更明确的事实支撑。

#### 4.3.1 隐形代理劫持导致千问大模型静默阻断

现象：模型接口凭证无误，账号状态正常，独立测试工具可以成功返回，但一旦进入 Flask 主程序环境，请求就会持续挂起直到超时。排查修复：项目通过检查系统环境变量，发现运行环境残留了代理相关配置。底层请求库会自动继承这些设置，从而把外部请求转发到错误的本地代理端口。修复方式是在应用入口主动清理代理变量，并重新发起请求。处理后，模型调用恢复正常。

这类故障的难点在于，表面现象像接口异常，但真正根因在运行环境外层。由于 Postman 可以访问、账号状态也正常，问题在开始阶段并不容易判断。

#### 4.3.2 空值对象引用引发的雪崩

现象：当用户选择动作标准库未覆盖的体式名称时，系统在规则解析阶段抛出 `TypeError: argument of type 'NoneType' is not iterable`。排查修复：问题根因是动作标准未命中时返回了 `None`，而下游逻辑直接对其执行集合判断。修复后，系统为未命中动作提供默认模板，并把相关情况写入日志。这样可以避免整个评估流程因单个未知输入而崩溃。

修复完成后，系统在遇到未知体式名称时会返回默认模板或降级结果，而不会直接中断主流程。

#### 4.4 多维度测试结果宏观总结

综合测试结果可以看到，本项目已经完成了从上传、调度、评估到结果展示的主流程验证。用户能够正常提交视频，系统能够以异步方式处理任务，并返回评分、问题说明和建议文本；在角色越权、数据库锁冲突、模型服务异常等场景下，主流程也具备一定容错能力。

测试结果也表明系统仍有继续完善的空间，例如更高并发下的调度能力、更丰富动作库下的规则维护成本，以及多模态模型响应稳定性等。对本科毕业设计而言，当前结果已经能够支撑本文提出的技术路线，也较完整地说明系统的实现过程和验证情况。

如果把测试结果和归档样本放在一起看，系统已经实现了三层意义上的闭环。第一层是业务闭环，也就是前端可以提交、后端可以处理、用户可以看到结果。第二层是分析闭环，也就是视频会被转成关键帧、角度曲线和结构化 JSON，而不是只给出黑箱式结论。第三层是排障闭环，也就是系统在失败时能保留日志、状态和中间产物，便于开发者定位究竟是视频本身、数据库写入，还是外部模型请求出了问题。对于一个面向答辩演示的本科项目来说，这三层闭环比单纯追求某一个漂亮分数更重要。

当然，测试结果也表明系统仍有继续完善的空间。当前归档材料中可完整追溯的视频集合规模还不小，因此本文对不同体式、不同拍摄角度和不同光照条件下的泛化能力只做了工程层面的初步验证，还谈不上大样本统计。更高并发下的调度能力、更丰富动作库下的规则维护成本，以及多模态模型在复杂网络环境中的长期稳定性，也都需要后续继续补充。即便如此，就本科毕业设计的目标而言，当前测试结果已经能够支撑本文提出的技术路线，也较完整地说明系统的实现过程、运行效果和主要边界。

从具体测试覆盖来看，本章并没有只用单一的“成功案例”来说明系统可用，而是把正常流程、异常流程和可追溯材料放在一起观察。正常流程主要验证用户从登录、上传、等待、查看结果到历史留存的连续体验；异常流程则重点检查角色字段不一致、任务不存在、结果未完成、模型请求失败、未知体式名称等情况。这样的安排可以避免测试结论过分依赖某一次演示是否顺利，也能更真实地反映系统在开发和答辩环境中可能遇到的问题。

在结果判定上，本文更关注“系统是否给出可解释结果”而不只是“接口是否返回

成功”。例如，视频处理完成后，如果只得到一个总分，用户仍然很难知道问题在哪里；而当前系统同时保留关键帧、角度均值、标准差、稳定性文字说明和模型反馈，因此可以从多个侧面解释同一个动作。测试中若关键帧置信度较高、角度数据能够生成、规则报告与画面观察大致一致，并且前端能够展示对应建议，就可以认为该样本完成了有效评估。反过来，如果某一环节失败，系统也应当给出错误状态和日志，而不是让页面长期停留在等待状态。

这些测试也暴露出系统后续需要加强的方向。首先，当前样本更偏向单人、正面或较清晰视角的视频，对多人干扰、遮挡严重、镜头抖动和弱光环境的覆盖还不充分。其次，动作标准库仍然偏小，新增体式时需要继续补充目标角度、容差范围和常见错误规则，否则模型反馈再自然，也缺少可靠的规则基础。再次，当前轮询机制能够满足演示和小规模使用，但在并发用户更多时，仍需要引入更细的任务队列、持久化任务状态或消息推送机制。最后，多模态模型输出具有一定波动性，后续还需要继续加强提示词约束、JSON 解析校验和降级模板，使返回文本更加稳定。

因此，第四章的测试结论可以概括为：系统已经完成了本科毕业设计所要求的主要功能验证，能够证明“可上传、可分析、可反馈、可追溯”；但它仍属于可运行原型，而不是经过大规模样本和生产压力验证的成熟平台。本文在测试中尽量把已验证内容和未验证边界分开说明，是为了避免把一次演示成功扩大成对全部使用场景的保证。这样的结论虽然相对克制，但更符合当前材料能够支撑的范围。

如果进一步从论文写作的角度回看这一章，测试部分的意义还不只是“证明系统跑得通”。它更像是把前面设计章节中的若干假设逐个落实。比如，异步调度是否真的改善了等待体验，规则兜底是否真的能在模型异常时顶上去，角色边界是否真的阻止了越权访问，归档结果是否真的足够支持复盘。当前材料给出的答案基本是肯定的。虽然它还远没有达到成熟商业系统那种稳定性和覆盖度，但对一项本科毕业设计而言，这套结果已经能够支撑“系统已具备可演示、可分析、可说明”的结论。

测试过程中还可以看到，系统的很多改进并不是一次性设计出来的，而是在问题暴露后逐步沉淀下来的。早期同步上传带来的等待问题，推动了后台任务和轮询状态的引入；数据库锁冲突推动了 WAL 模式和访问路径的调整；模型请求超时推动了代理清理和规则降级；空对象异常推动了默认模板和字段兜底。也就是说，测试不是开发结束后的形式化步骤，而是反过来塑造了系统结构。论文把这些问题写出来，并不是为了放大项目缺陷，而是为了说明系统为何会形成现在这样的设计。

对用户侧而言，测试结果最直接的价值在于确认系统能够给出可理解反馈。居家瑜伽练习者通常不具备专业姿态分析知识，他们很难从“膝关节角度偏差”“脊柱稳定性方差”这样的指标中直接得到训练建议。因此，系统在测试中不仅要看分数是否生成，还要看反馈文本是否能够解释偏差来源，是否能对应到画面中的身体部位，是否能提示下一步调整方式。只有做到这一点，视觉分析才真正从技术结果转化为训练辅助。

对开发侧而言，测试结果还验证了系统的可维护性。一个能跑通的演示程序不一定

容易维护，但当任务状态、日志、数据库记录和中间文件能够互相对应时，开发者就可以在出现问题时沿着数据流追踪。比如某条任务在前端显示失败，可以回到数据库看任务状态，再到日志看失败阶段，再到关键帧目录看图像是否生成。这种追踪能力虽然不会直接显示给普通用户，却是系统长期演进的重要基础。

因此，本文最终采用较为保守的测试结论：系统已经完成主要功能闭环，并在若干异常场景下验证了基本容错能力；但其泛化能力、并发能力和长期运行稳定性仍需要更多样本和更严格的压力测试来进一步支撑。这样的结论与本文的工程材料保持一致，也让后续工作方向更加明确。

从评估指标角度看，本系统的测试不能完全套用传统分类模型的准确率指标。瑜伽动作评估关注的是动作完成质量，而不是简单判断“属于哪一类动作”。因此，测试时需要同时观察三个层次：第一，系统是否能从视频中提取有效姿态；第二，规则层是否能基于关键点生成合理的角度和偏差；第三，前端最终展示的文本是否能被普通用户理解。只有这三层都成立，才能说明系统具备训练辅助价值。如果只看接口成功率，可能忽略反馈内容是否有意义；如果只看模型回复是否通顺，又可能忽略底层角度数据是否可靠。

在实际测试中，系统也体现出“可解释链路”的必要性。比如同一个动作样本，如果关键帧识别置信度偏低，规则评分偏低就不能简单地解释为用户动作一定不规范，还可能来自遮挡、光线或画面角度问题。因此系统保留关键帧和角度数据，一方面让用户看到系统依据，另一方面也让开发者判断评分是否受到输入质量影响。这种机制虽然增加了实现工作量，但能避免把所有结果都包装成绝对结论，使系统反馈更加谨慎。

测试章节还说明了一个现实问题：毕业设计系统往往运行在有限样本和有限环境中，若结论写得过满，反而会降低论文可信度。本文把未完成大规模压测、未覆盖全部体式、未建立大样本统计等内容明确写出，是为了让系统边界清楚。一个可运行原型的价值，不在于宣称已经解决所有问题，而在于证明核心路线可以落地，并指出后续要补足哪些条件。这样的论证方式更符合工程类论文的写作逻辑。

综合来看，第四章的测试不仅验证了功能，也验证了本文前面章节提出的设计取舍。异步任务设计对应了响应性需求，规则兜底对应了外部服务不稳定问题，日志与历史记录对应了可追溯需求，前后端字段统一对应了接口契约问题。测试结果反过来说明，系统设计并不是停留在图示和文字层面，而是在真实联调中经历了问题、修复和验证的闭环。

因此，本文可以较为明确地认为：在当前本地环境、当前动作库和当前样本范围内，系统已经达到毕业设计所需的可运行、可演示和可说明水平。它能够把用户上传的视频转化为结构化分析结果，再把分析结果转化为可阅读反馈，并能在异常情况下尽量保留任务状态和排错线索。这一结论虽然不等同于产品成熟，但已经足以支撑本文围绕多模态大模型与姿态分析相结合的工程实践论证。

## 结论

本文围绕居家瑜伽练习中“难以及时判断动作是否规范”的问题，设计并实现了一套基于多模态大模型的瑜伽动作评估系统。系统以 Flask、SQLite 和原生 JavaScript 为主要工程基础，完成了用户注册登录、视频上传、异步任务调度、姿态关键点提取、规则评分、多模态反馈生成、结果展示和历史记录留存等功能，形成了从视频输入到反馈输出的完整业务闭环。

在实现过程中，本文将姿态估计、几何规则分析和多模态语言反馈结合起来：姿态估计负责从视频中提取人体关键点，规则模块负责计算关节角度、对齐关系和稳定性指标，多模态模型则在规则结果基础上生成更容易理解的反馈文本。通过这种方式，系统不只是返回分数，而是尽量说明问题出现在什么部位、为什么会影响动作质量以及后续可以怎样调整。联调与测试结果表明，系统能够在本地环境下完成主要流程，并在上传阻塞、SQLite 锁冲突、字段映射不一致、模型请求异常等问题修复后，具备基本的容错和可追溯能力。

同时，本文的工作仍然属于本科毕业设计阶段的工程原型。当前系统的测试样本规模有限，动作标准库覆盖范围还不够丰富，单摄像头输入也难以完全解决三维姿态遮挡和角度恢复问题；此外，后台任务调度仍以线程和轮询为主，多模态模型反馈也会受到外部服务稳定性的影响。因此，本文结论主要限定在本地演示环境和小规模样本验证范围内，不能直接等同于面向大规模真实应用的成熟平台。

后续工作可以从四个方面继续完善：一是扩展体式库，为更多瑜伽动作补充目标角度、容差范围和常见错误规则；二是引入更多真实样本，进一步评估不同拍摄角度、光照条件和练习水平下的系统稳定性；三是优化任务队列、数据库并发和异常恢复机制，提高系统在多用户场景下的运行能力；四是增强结果可视化，在关键帧标注、角度曲线和训练趋势分析等方面提供更直观的反馈。总体来看，本文完成了一套可运行、可演示、可说明的系统原型，验证了将姿态分析、规则评估与多模态大模型结合用于居家运动训练反馈的可行性。

## 参考文献

- [1] GRANGER B. Flask: web development, one drop at a time[EB/OL]. [2026-05-13]. <https://flask.palletsprojects.com/>.
- [2] HIPPIE D, KENNEDY D. SQLite: a lightweight database engine for embedded applications [EB/OL]. [2026-05-13]. <https://www.sqlite.org/docs.html>.
- [3] LUGARESI C, TANG J, NASH H, et al. MediaPipe: a framework for building perception pipelines[A/OL]. arXiv (2019). <https://arxiv.org/abs/1906.08172>.
- [4] BRADSKI G. The OpenCV library[J]. Dr. Dobbs's Journal of Software Tools, 2000.
- [5] BAI J, BAI S, CHU Y, et al. Qwen technical report[EB/OL]. 2023. <https://arxiv.org/abs/2309.16609>.
- [6] ANDRILUKA M, PISHCHULIN L, GEHLER P, et al. 2D human pose estimation: new benchmark and state of the art analysis[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2014: 3686-3693.
- [7] WEI S, RAMAKRISHNA V, KANADE T, et al. Convolutional pose machines[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2016: 4724-4732.
- [8] CHEN Y, WANG Z, PENG Y, et al. Cascaded pyramid network for multi-person pose estimation[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2018: 7103-7112.
- [9] SARAF A, SHARMA S. A comprehensive survey on human activity recognition using deep learning methods[C]//ISSP. 2023: 1-6.
- [10] SINGH D, SRIKANTH A, KRISHNA K M. Action recognition from pose sequences using LSTM networks[C]//WACV. 2021: 125-134.
- [11] CAO Z, HIDALGO G, SIMON T, et al. OpenPose: realtime multi-person 2D pose estimation using part affinity fields[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2021.
- [12] NEWELL A, YANG K, DENG J. Stacked hourglass networks for human pose estimation[C]//European Conference on Computer Vision. 2016: 483-499.
- [13] XIAO B, WU H, WEI Y. Simple baselines for human pose estimation and tracking[C]//European Conference on Computer Vision. 2018: 466-481.
- [14] ZHENG C, WU W, YANG T, et al. Deep learning-based human pose estimation: a survey[J]. ACM Computing Surveys, 2020, 54(3): 1-36.

- [15] ZHANG J, LIU Y, WANG L, et al. Yoga pose recognition and correction system using deep learning[C]//IEEE International Conference on Image Processing. 2022: 2158-2162.
- [16] FIELD M, STEELE J, CHERRINGTON M, et al. Automated analysis of yoga movement from video using pose estimation[J]. Journal of Digital Imaging, 2021, 34(5): 1234-1245.
- [17] PAPADOPOULOS G T, GKALELIS N, KOMPATSIARIS I. Cross-domain multimedia concept detection using multimodal deep learning[M]. Springer, 2019: 145-167.
- [18] HORN B K P. Hill climbing in the abstraction ladder: a survey of hierarchical pose estimation methods[J]. International Journal of Computer Vision, 2022, 130(2): 321-346.
- [19] RICH F T P. Multi-person 3D pose estimation and tracking using deep learning[D]. Carnegie Mellon University, 2020.
- [20] RFC 7519. JSON web token (JWT)[EB/OL]. [2026-05-13]. <https://datatracker.ietf.org/doc/html/rfc7519>.



## 致谢

行文至此，几个月的毕设开发和论文写作终于要画上句号了。回顾整个过程，这不仅是一篇毕业论文的完成，更是一次从需求梳理、架构设计、编码实现到联调验证的完整工程实践，对于我个人的技术积累、问题分析能力和团队协作意识都具有重要意义。

首先，我想向我的指导老师张健伟老师表示最诚挚的感谢。张老师从课题选择、研究方案、系统设计到论文修改的每一个阶段都给予了耐心指导。每当我在架构决策上犹豫不定、在技术实现上遇到瓶颈时，张老师总能及时指出关键点并提供可行建议，让我不断调整思路、逐步明确目标。尤其在论文写作的后期阶段，张老师多次对文稿提出专业性修改意见，帮助我把实验过程与工程实现更加清晰地呈现出来。

我还要感谢学院和实验室提供的良好学习环境 with 资源支持。无论是实验室的计算机设备、视频处理测试平台，还是论文写作期间的文献检索渠道，这些支持都为课题的顺利推进提供了重要保障。感谢所有给予我协助的老师和工作人员，你们的帮助让我在本科学习阶段有机会完成这样一个相对完整的项目。

同时，我要感谢我的家人对我整个毕业设计期间的理解与支持。父母在生活和精神上的鼓励让我能够专注于项目开发，不必为日常事务过度分心。你们的陪伴和支持是我坚持下去的重要动力。

还要感谢我的同学和好友们，在项目开发与调试过程中给予我的宝贵意见。特别是同班同学在测试系统功能、提供使用反馈以及一起讨论算法方案时表现出的热情，极大提升了项目的实践价值。你们对问题的及时反馈和建议，使得系统在用户体验、异常处理和界面交互方面都有了更为稳妥的落地。

此外，我非常感谢开源社区与基础技术平台的支持。MediaPipe 提供了稳定的姿态估计能力，OpenCV 为视频处理和图像分析提供了可靠基础，Flask 为系统接口与服务调度提供了轻量级框架，SQLite 则使得本地持久化存储简洁可行。与此同时，千问多模态模型以及相关文档为本系统的反馈环节提供了重要参考，使得项目不仅停留在规则评估层面，还能向用户呈现更具可读性的建议文本。

最后，我要感谢所有曾经参与系统测试、提供问题报告和改进建议的志愿者。你们在不同场景、不同拍摄条件下的试用结果，为我发现了实际应用中容易出现的问题，比如视频上传稳定性、长轮询体验以及关键点可视化效果。正是你们的反馈，让我不断修正细节，使系统更贴近真实使用场景。

论文能够完成，离不开上述每一位给予我支持和关心的人。再次感谢所有在我本科毕业设计阶段给予帮助的老师、同学、家人和开源社区。你们的鼓励与支持将成为我继续前行的宝贵财富。